INSY7315 INFORMATION SYSTEMS 3E (WIL) · TASK 3

SizaniConnect: Final Report

Food relief operations for Sizani Community Food Network NPC. The final system, the changes since Task 1 and Task 2, and how the lecturer feedback was applied.

Institution	The Independent Institute of Education, Varsity College
Module	INSY7315 Information Systems 3E (Work Integrated Learning)
Submission	Task 3: final project report, individual submission
Group	Group 13: Sizani Community Food Network NPC (SizaniConnect)
Member	Byron Bousfield, ST10445541 (sole member of the group)
Client	Sizani Community Food Network NPC, a simulated case-study client
Release described	SizaniConnect v1.1.0
Date	30 October 2026

Sizani Community Food Network NPC is a case-study client constructed for this project, as allowed for Simulation-type WIL. No real organisation was interviewed, and every person, household and donor in the system's data is synthetic.

Contents

1. About this submission....4
1.1 Group and members....4
1.2 Links....4
1.3 What this submission contains....5
2. Improvements based on lecturer feedback....6
2.1 Improvements found during Task 3....8
3. Changes since Task 1 and Task 2....10
3.1 Release history....10
3.2 Design decisions changed during implementation....10
3.3 Everything that changed in v1.1.0....12
4. Requirements as delivered....14
4.1 Scope....14
4.2 User roles....14
4.3 User stories....15
4.4 Acceptance criteria....16
4.5 Business rules....17
4.6 User experience journey map....18
4.7 Non-functional requirements: how each was verified....20
5. Project plan for Task 3....22
5.1 Work breakdown structure....22
5.2 Contention and how it was resolved....23
6. Analysis and design....25
6.1 Domain model....25
6.2 Design class diagram....25
7. Behaviour: sequence and state diagrams....28
7.1 Sequence diagrams....28
7.2 State diagrams....37
8. Data schema....41
8.1 Technology....41
8.2 Entity relationship diagrams....41
8.3 Schema design decisions....45
8.4 Migrations and how each is undone....46
9. Architecture and security....48
9.1 Architecture....48
9.2 Design patterns, and where to find them....49
9.3 Cloud architecture as deployed....49
9.4 Threats and controls....51
9.5 Authentication, sessions and recovery....52
9.6 POPIA....53
10. Testing and quality....54
10.1 Test levels....54
10.2 Coverage....54
10.3 Performance....55

11. DevOps, deployment and rollback....56
11.1 Branching and version control....56
11.2 Continuous integration and deployment....56
11.3 Environments and rollout....57
11.4 Rollback....58
11.5 When something fails in production....58
12. Running costs....60
13. Change management and handover....62
14. Reflection and limitations....63
14.1 What is not done, and what would come next....63
14.2 What I learned....63
References....64
Annexure A. Declaration of Authenticity....65
Annexure B. Peer evaluation....66
Annexure C. User guide....67
Annexure D. EXPO materials....68
Annexure E. Glossary....69

1. About this submission

This report describes SizaniConnect as it stands at release v1.1.0, the version handed in for Task 3. It does three things the brief asks for. It shows how each point of lecturer feedback on Task 1 and Task 2 was applied (Section 2). It records every change made since the Task 1 design and the Task 2 release, with the reason for each (Section 3). And it brings the full Task 1 documentation up to date so that it matches the final system exactly (Sections 4 to 14). Where the built system differs from what Task 1 planned, the text says so and says why, rather than quietly describing the plan.

1.1 Group and members

Table 1. Group details

Item	Detail
Group	Group 13
Group name (client)	Sizani Community Food Network NPC: the system is SizaniConnect
Members	**Byron Bousfield, ST10445541.** Sole member: no other students were allocated to Group 13, and the lecturer was informed by email. Every role in the project (analysis, design, development, testing, DevOps, documentation, EXPO) was carried out by this one member.
Client	Sizani Community Food Network NPC, a Johannesburg non-profit running a central depot and six distribution sites. The client is simulated and declared as such (Annexure A).
Weekly registers	Submitted weekly as the group register PDF, as the module requires.

1.2 Links

Table 2. Links to the system and its source

What	Where
Live system (production)	`https://sizaniconnect-bb7315.azurewebsites.net`
Staging	`https://sizaniconnect-bb7315-staging.azurewebsites.net`
Source code (GitHub)	`https://github.com/ByronBousfield1/sizaniconnect`
This release	`https://github.com/ByronBousfield1/sizaniconnect/tree/v1.1.0` (tag v1.1.0 on `main`)
CI/CD runs	`https://github.com/ByronBousfield1/sizaniconnect/actions`
Demo accounts	coordinator@sizani.test, volunteer@sizani.test, ops@sizani.test, donor@sizani.test, admin@sizani.test, all with the password **Sizani-Demo-2026**

> **Note.** The Task 1 prototype was a design-only prototype. It has been superseded by the working system above, which implements every screen from the journey map in Section 4.6.

1.3 What this submission contains

Table 3. Parts of the individual submission

Part	Where to find it
Final report (this document)	Sections 1 to 14 and the annexures
Improvements based on lecturer feedback	Section 2
User guide	Separate document, *SizaniConnect User Guide* (55 pages, every feature by role, annotated screenshots, setup, FAQ and troubleshooting). Summarised in Annexure C and included after this report in the combined submission.
Walkthrough video	*SizaniConnect walkthrough* (MP4, about four and a half minutes): each role's main journey on desktop and phone
EXPO PowerPoint and display board	Annexure D
Declaration of Authenticity	Annexure A
Peer evaluation	Annexure B
Running the system yourself	User guide Appendix A, and the README in the repository

2. Improvements based on lecturer feedback

Every point raised in the feedback on Task 1 and Task 2 was treated as a requirement for Task 3. Each was turned into a change to the system, a change to the documentation, or both, and each change has an automated test or a section of this report that shows it. Table 4 is the summary; the paragraphs after it explain what was done for each point.

Table 4. Lecturer feedback and how it was applied

#	Feedback	What changed	Evidence
F1	Task 1, requirements: acceptance criteria for invalid or corrupted QR payloads, on low-end hardware (US-09)	Acceptance criteria AC-09.1 to AC-09.7 written. Household cards with a QR code that carries its own check character. The event desk checks every scan before sending it and explains each kind of bad scan. Camera scanning tuned for cheap phones.	Section 4.4; SD-02; desk state diagram (Section 7.2); `desk.spec.ts`, `domain.test.ts`
F2	Task 1, work breakdown: show resource contention and release-candidate milestones	The Task 3 plan is levelled for one resource and names the points where tasks compete for that one person, with release-candidate milestones RC-1 to RC-3.	Section 5
F3	Task 2, sequence diagrams: alternative paths such as a network timeout during the TLS handshake	New sequence diagram SD-02b with three alternatives. The browser now gives up after 20 seconds, retries reads once, never retries writes blindly, and shows a connection banner. A repeated collection after a lost answer is recognised as already saved.	Section 7.1; `network.test.ts`, `resilience.spec.ts`
F4	Task 2, state diagrams: failure states such as storage full and database connection lost, with recovery transitions	New system availability state diagram and event desk state diagram. The API answers each failure with a specific 503 and says whether anything was saved; the health check reports ok, degraded or down; the pool recovers without a restart.	Section 7.2; `failures.test.ts`; runbook in Section 11.5
F5	Task 2, deployment: rollback steps for failed destructive migrations	Every migration after the first has a down script. A destructive migration backs up what it changes. A rollback command, a switch that stops a restart re-applying a migration, and a rehearsal in CI before every deployment.	Sections 8.4 and 11.4; `migrations.test.ts`
F6	Task 2, back-end security: make sure cellphone numbers are validated	One parser for South African cellphone numbers in the browser and the API, stored in one format (+27...), and enforced by the database (migration 002).	BR-11 (Section 4.5); `domain.test.ts`, `households.test.ts`, `phone.test.tsx`
F7	Task 2, back-end security: what happens if the administrator forgets the password? Can they reset themselves?	Recovery codes for every user, password change while signed in, administrators sending each other reset links, and an audited break-glass command for the case where nothing else is left.	SD-04; Section 9.5; `account-recovery.test.ts`

#	Feedback	What changed	Evidence
F8	Task 2, database: what happens when a user retires? How are they removed completely, with the administrator managing it?	Offboarding (account stops at once, reversible for 30 days) followed by erasure (deleted outright, or anonymised where history must still point at someone). Archived households are anonymised after 24 months (NFR-16).	SD-05; user state diagram; Section 8.4; `retention.test.ts`

F1. Invalid or corrupted QR payloads on low-end hardware

Task 1 asked for collection by scanning a household's reference code (US-09) but never said what a scan contains or what happens when it is wrong. A household card now carries a QR code holding `SZN1:<reference code>:<check character>`. The prefix identifies a SizaniConnect card, so any other QR code is recognised as foreign. The check character is a Luhn mod 32 digit over the code, which catches every single wrong character and most swapped pairs that slip past the QR code's own error correction, for example when a cheap USB scanner drops a character into the input box. The code uses only upper-case letters, digits, a colon and a hyphen, so the QR code fits in alphanumeric mode at error-correction level M: small, with coarse modules that a low-cost phone camera can read.

The same parser runs in the browser and in the API. A damaged, foreign, over-long or half-typed scan is explained in plain language and never sent to the server, and after two failures the desk suggests typing the code printed under the QR code. Camera scanning uses the phone's built-in barcode detector where it exists and otherwise loads a small decoder only when the camera is opened; it reads at 640 by 480 pixels, decodes four frames a second, and gives up after 20 seconds with a clear message. The end-to-end suite prints a real card, turns it into a fake camera feed, and reads it on a 360-pixel phone with the processor slowed six times: about two seconds against a limit of five.

F2. Resource contention and release-candidate milestones

Group 13 has one member, so every task competes for the same person. The Task 3 plan in Section 5 is levelled: no two tasks overlap, and the places where work could have been done in parallel by a larger team are named as contention points. Release candidates mark the points where the system is frozen before work moves on: RC-1 when the Task 2 presentation was finished and Task 3 started, RC-2 the code freeze before documentation, and RC-3 the release of v1.1.0 to production before the report is handed in.

F3. Network timeouts during the TLS handshake

Distribution sites rely on mobile data. A request can fail at three different points, and each needs a different answer, so SD-02b in Section 7.1 shows all three: the TLS handshake never completes (nothing reached the server, so nothing was saved); the server saved the collection but the answer was lost on the way back; and the server could not reach its database. The browser abandons any request after 20 seconds, retries reads once automatically, and never retries a write by itself. At the event desk a repeated collection is safe: the database accepts only one collection per household per event, and when the repeat comes from the same volunteer the desk reports it as already saved rather than as a duplicate.

F4. Failure states and recovery transitions

Two new state diagrams in Section 7.2 show what the system does when things go wrong and how it gets back. The system availability diagram covers the database becoming unreachable, recovering, running out of storage (Azure then switches the server to read-only), being overloaded, and the client

losing its connection. Each failure produces a specific 503 answer with a Retry-After header and a message that says whether anything was saved: DATABASE_UNAVAILABLE, STORAGE_FULL, DATABASE_BUSY or DATABASE_TIMEOUT. None produces a 500. The connection pool replaces broken connections by itself, so no restart is needed once the database is back. The integration tests put the real application into each state and back out again.

F5. Rollback for failed destructive migrations

Each migration runs in its own transaction, so one that fails while deploying leaves no trace and is not recorded as applied. For a migration that succeeded but must be undone, every migration after the first now has a down script. A destructive migration, such as 002, which rewrites every stored cellphone number, first copies the original values into a backup table in the same transaction, and its down script restores them, skipping any row edited since. The database command-line tool reports status and rolls back to a named version, newest first, one transaction per step, and refuses to start if a down script is missing. A setting (`SKIP_MIGRATIONS`) stops the next restart re-applying a migration that was just undone. CI rehearses the whole cycle on a populated database before every deployment. Section 11.4 gives the step-by-step procedure.

F6. Cellphone validation

One parser accepts the ways people actually write South African cellphone numbers (082 555 0134, 082-555-0134, +27 82 555 0134, 27825550134) and stores a single form, +27825550134. Landlines, toll-free and share-call numbers and wrong lengths are refused with a message that says why. The browser checks the field when the user leaves it, the API checks it again, and the database refuses anything else through CHECK constraints on both users and households.

F7. An administrator who forgets the password

Email reset (US-20) depends on email being configured and on the person still having access to the mailbox. Four routes now exist, in order of preference: the emailed link; one of ten single-use recovery codes the user created in advance (only keyed hashes are stored); a reset link sent by another administrator; and, when there is no other administrator and no codes, a break-glass command run from the Azure portal’s SSH console that prints a 30-minute link and writes BREAK_GLASS_RESET to the audit log. Administrators are reminded on their start page until they have created recovery codes. SD-04 shows the recovery-code path.

F8. Removing a user who retires

Leaving is two deliberate steps for the administrator. **Offboard**: the account stops working at once; sessions, reset links and recovery codes are cancelled; future shift places are freed; and the administrator is warned if a site is left without a coordinator. This is reversible for 30 days. **Erase**: only after offboarding, and confirmed by typing the email address. An account that never did anything is deleted outright. One that did (signed in, recorded collections, worked shifts) keeps an anonymous row called “Former user”, because those records and the append-only audit trail must still point at something; the row holds no name, email, cellphone or password. The same Data retention screen lists households archived for more than 24 months and anonymises them with one audited action, which implements NFR-16. SD-05 and the user state diagram show the flow.

2.1 Improvements found during Task 3

Writing the user guide, checking every requirement for this report and running the tests repeatedly turned up problems that no feedback had pointed at. They were fixed in the same release:

- **Coordinators could change another site’s households.** A site coordinator could edit, suspend or resubmit a household registered at a different site. They can still read every household (to

spot duplicates and answer questions), but changes now return 403 unless the household belongs to their site, and the buttons are hidden (threat T-04).

- **Search terms and link tokens reached the request log.** Request logs held the full address and headers, so a household search for a name, or the token in an activation or reset link, was written to the log. Logs now keep the method and path only (NFR-14).
- **A form could lose a value when several fields were filled at once.** Found while investigating an intermittent end-to-end failure: each field’s change handler copied the form as it was before the others changed, so a password manager filling an email and a password in one go could leave one blank. Every form now updates from its latest values.
- **Archiving was only reachable through the API.** BR-08 (archive after 12 months without a collection) now has a button on the Households screen with a confirmation that explains the consequences.
- **Form fields did not line up.** A hint under one field pushed its input below its neighbours. Fields in a row now share label, hint, input and error rows.
- **Browser, accessibility and load testing were missing.** End-to-end tests with WCAG 2.1 AA checks now run in CI on every push, and a load test measures NFR-01 to NFR-03 (Section 10).
- **Dependencies were behind.** All open Dependabot updates were applied, including the GitHub Actions that warned about Node 20.

3. Changes since Task 1 and Task 2

3.1 Release history

Table 5. Releases

Release	Date	What it contained
v1.0.0	5 Oct 2026	Task 2 submission. All 20 user stories, the ten business rules, CI/CD with GitHub Actions, staging and production on Azure.
v1.0.1	14 Oct 2026	A request ceiling on every route (raised by CodeQL), the Azure setup script brought in line with the deployed environment, and the Task 2 presentation.
v1.1.0	30 Oct 2026	Task 3. Everything in Section 2: the eight feedback items and the improvements found during Task 3 (Section 2.1), including browser and accessibility tests, the load test and dependency updates. Four database migrations in total, each new one with a down script.

3.2 Design decisions changed during implementation

Task 1 was a plan. Building the system showed where the plan was wrong, too expensive, or impossible on the subscription available. Each change below kept the requirement the Task 1 design was meant to meet; only the means changed.

Table 6. Design decisions changed since Task 1

Area	Task 1 design	Final system	Why
Back-end stack	ASP.NET Core 8, Entity Framework Core, xUnit, FluentValidation	Node.js 22 with TypeScript, Express 5, `pg` with hand-written parameterised SQL, Vitest, Zod	The layered architecture, patterns and API contract were kept. TypeScript at both ends meant one language, shared types and one toolchain for a one-person team. The design class diagram maps one-to-one onto the TypeScript classes.
Unit of Work	EF Core DbContext	`UnitOfWork.run()` wrapping a PostgreSQL transaction and handing out repositories bound to it	Same pattern without an ORM.
Front-end hosting	Azure Static Web Apps, separate from the API	The React build is served by the API's App Service, same origin	Static Web Apps' free tier cannot proxy to an App Service API. One origin keeps the refresh cookie SameSite=Strict, removes CORS, and lets the CSP be `self` everywhere.
Staging	App Service deployment slot with swap	A second web app on the same B1 plan	Slots need the Standard tier; a second app on the same plan costs nothing extra. Rollback redeploys the previous release tag (Section 11.4).

Area	Task 1 design	Final system	Why
Database networking	Private subnet and private endpoint	Public endpoint, TLS required, firewall admitting only the two web apps' outbound addresses	Every private DNS zone name was rejected on the Azure for Students subscription. Moving to private networking later needs no code change, only a new connection string.
Migrations	Pipeline step before deployment	Applied by the app at startup under an advisory lock; down scripts and a rollback command added in v1.1.0	The pipeline never needs network access to the database, so the firewall stays closed to it.
Secrets	Azure Key Vault with managed identity	App Service application settings (encrypted at rest); deployment credentials as GitHub secrets	Same effect for one application at lower setup cost. Key Vault references can be added without code changes.
File storage	Blob Storage for delivery notes and signatures	Deferred; donations have a notes field	No Must story depends on uploads. US-12's "attach the delivery note" is the one part of a story not delivered (Section 4.3).
Monitoring	Application Insights	App Service log stream with structured JSON logs, App Service health check, Azure Monitor storage alert	Enough at this scale and free. Application Insights is costed for larger scales in Section 12.
Household status	Draft, Pending, Active, Suspended, Archived	The same five. Registration goes straight to Pending; Draft is reached by returning a household for correction. Since v1.1.0 an archived household is also marked anonymised once its personal data is removed	Matches SD-01. Anonymisation implements NFR-16.
Reference code	`SZN-2026-04817` (digits)	`SZN-2026-7K4QX2` (Crockford base32), printed with a QR code since v1.1.0	Random rather than sequential, so a code says nothing about the size of the register (T-12), and no letters that are confused when read aloud.
Identity number hash	Salted SHA-256	HMAC-SHA256 with a secret key held outside the database	A plain salted hash of a 13-digit ID number with a known structure can be brute-forced; a keyed hash cannot without the key.
Stock ledger	Stock derived only by summing movements	`quantity_remaining` on each batch with a CHECK (>= 0) and row locks, plus the append-only movement ledger	Makes over-issuing impossible under concurrency; the ledger still explains every change.
Schema additions	—	Refresh tokens, reset tokens, recovery codes, parcel components, user site and donor links, event minimum volunteers and variance, user lifecycle and anonymisation columns	Needed by the authentication design, BR-05, BR-07, coordinator site scope, the donor portal and the feedback items.

Area	Task 1 design	Final system	Why
Separation of duties	Not specified	The person who registered a household cannot verify it	A fraud control alongside BR-03.
Rate limiting	10 sign-ins a minute per address	That, plus 1,000 requests a minute per address on every route	Raised by CodeQL after the first push; closes the denial-of-service gap (T-10).
Test levels	Unit, integration, Playwright end-to-end against staging	Unit, integration against real PostgreSQL, component, and Playwright end-to-end with accessibility checks in CI against a fresh database, plus a load test	Running end-to-end tests in CI on every push catches problems before staging rather than after.

3.3 Everything that changed in v1.1.0

Each change was made on its own branch from `develop`, tested in CI, and merged with a merge commit so the history shows which branch delivered what.

Table 7. Branches merged for v1.1.0

Branch	Change	Reason
`chore/dependency-updates`	Express 5.3, React 19.3, Vite 6.4, nodemailer 10.2, GitHub Actions v7	Open Dependabot updates; Node 20 deprecation warnings
`feature/migration-rollback`	Down scripts, `db status / migrate / rollback`, `SKIP_MIGRATIONS`, rehearsal in CI, deploy of an earlier tag	F5
`feature/cellphone-validation`	Cellphone parser in browser and API, migration 002 with backup table and CHECK constraints	F6
`feature/account-recovery`	Recovery codes (migration 003), change password, admin reset links, break-glass command, My account page	F7
`feature/offboarding-and-retention`	Offboard, erase, Data retention screen, household anonymisation (migration 004)	F8, NFR-16
`feature/qr-household-cards`	Printable household card with QR code, scan checking, camera scanning, Permissions-Policy header	F1, US-09
`feature/failure-handling`	Specific 503 answers, pool time-outs, health states, browser time-outs and retries, connection banner, runbook	F3, F4
`test/e2e-accessibility`	26 Playwright journeys with axe-core WCAG 2.1 AA checks at 1280 and 360 px, run in CI	NFR-10, NFR-12, AC-09.x
`test/performance`	Load test for NFR-01 to NFR-03 and recorded results	NFR-01 to NFR-03
`fix/form-alignment`	Fields in a form row share label, hint, input and error rows	Usability

Branch	Change	Reason
`feature/archive-inactive-button`	Archive inactive households button with confirmation	BR-08
`fix/coordinator-site-scope`	Coordinators change only their own site’s households	T-04
`fix/no-query-strings-in-logs`	Request logs keep the path only	NFR-14
`fix/fast-typing-lost-input`	Forms keep every value when several fields are filled at once; an end-to-end test waits for the page it fills	Reliability

4. Requirements as delivered

This section restates the Task 1 requirements as they stand in the final system: what was delivered, what changed, and what was added in response to feedback. Sizani's problem has not changed. Paper registers, WhatsApp shift lists and delivery-note spreadsheets let a household collect at two sites in one cycle, hide what stock is on hand, cost coordinators hours a week, and make donor reporting slow. SizaniConnect replaces them with one record, captured where the work happens, that enforces the eligibility and one-parcel rules at the moment of capture.

4.1 Scope

Table 8. Scope delivered

Task 1 scope item	Status in v1.1.0
Household registration, verification, search and lifecycle, including members	Delivered, plus printable household cards (US-24) and anonymisation after 24 months (NFR-16)
Distribution sites and event scheduling	Delivered
Collection capture with one parcel per household per event	Delivered, enforced by the database; QR scanning added (US-09, F1)
Volunteer shifts: publish, sign up, check in, hours	Delivered
Donors and donations, cash and in kind, with receipt reference	Delivered, except attaching a scanned delivery note (deferred with Blob Storage)
Stock items, batches with expiry, first-expiry-first-out	Delivered
Operational and donor reporting with export	Delivered: monthly report on screen and as CSV; donor impact page and printable receipts. PDF export is by the browser's print to PDF
Role-based access, authentication and a POPIA audit trail	Delivered, plus account recovery (US-21), offboarding and erasure (US-22, US-23)

Out of scope, as in Task 1: payment processing, payroll and accounting, a native mobile app (the web app works on a 360-pixel phone instead) and SMS notifications. Offline capture remains out of scope; Section 7.1 shows how the system now behaves when the connection drops instead.

4.2 User roles

Table 9. User roles

Role	Responsibilities in the final system
Administrator	Creates accounts and assigns roles; sends reset links; offboards leavers and erases their personal data; anonymises long-archived households; manages sites; reviews the audit log.
Operations manager	Verifies, returns, suspends, reinstates and archives households; schedules events and their shifts; captures donations; manages stock and write-offs; runs the monthly report.

Role	Responsibilities in the final system
Site coordinator	Registers households and prints their cards; runs the event desk; opens, closes and reconciles events at their own site; approves a second parcel with a reason. Can read every household but change only those at their own site.
Volunteer	Signs up for and withdraws from shifts, checks in, and records collections at the event desk. Sees only what the desk needs: no address or phone number.
Donor	Sees their own donations, what the goods reached, and printable receipts.
Every user	Signs in and out; resets a forgotten password; creates recovery codes; changes their password.

4.3 User stories

US-01 to US-20 are the Task 1 stories. US-21 to US-24 were added in Task 3 in response to the feedback.

Table 10. User stories and their status

ID	Role	Story (shortened)	Pri.	Status	Screen
US-01	Coordinator	Register a household with its members	Must	Delivered	S4
US-02	Coordinator	Be warned when a household is already registered	Must	Delivered	S4
US-03	Ops manager	Verify or return a pending household	Must	Delivered	S5
US-04	Coordinator	Search by name, reference code or suburb	Must	Delivered	S5a
US-05	Ops manager	Schedule a distribution event	Must	Delivered	S11
US-06	Volunteer	See shifts that still need people	Must	Delivered	S10
US-07	Volunteer	Sign up for and withdraw from a shift	Must	Delivered	S10
US-08	Volunteer	Check in at a shift	Must	Delivered	S10
US-09	Volunteer	Record a collection by scanning the reference code	Must	Delivered; QR cards and camera scanning added in v1.1.0	S6, S7
US-10	Volunteer	Be stopped from issuing a second parcel	Must	Delivered	S7
US-11	Coordinator	Close an event and reconcile stock	Must	Delivered	S8a
US-12	Ops manager	Capture a donation and attach the delivery note	Must	Delivered except the attachment	S12
US-13	Ops manager	Issue stock oldest expiry first	Must	Delivered	—
US-14	Ops manager	Be alerted to low stock and batches near expiry	Should	Delivered as an alerts panel	S9, S13
US-15	Ops manager	Monthly report by site	Must	Delivered	S15

ID	Role	Story (shortened)	Pri.	Status	Screen
US-16	Donor	See what my donations funded	Should	Delivered	S16
US-17	Donor	Download my receipt reference	Should	Delivered (printable receipt)	S17
US-18	Administrator	Create users and assign roles	Must	Delivered	S20
US-19	Administrator	Review the audit trail	Must	Delivered	S22
US-20	Every user	Reset my own password by email	Must	Delivered	S2, S2c
US-21	Every user	As a user, I want single-use recovery codes and to change my password while signed in, so that I can get back in even when email is not available	Must	New in v1.1.0	S24, S2b
US-22	Administrator	As an administrator, I want to offboard someone who leaves, so that their access stops at once without losing the history of what they did	Must	New in v1.1.0	S20
US-23	Administrator	As an administrator, I want to erase a leaver’s personal data and anonymise long-archived households, so that we keep personal information no longer than we need it	Must	New in v1.1.0	S23
US-24	Coordinator	As a site coordinator, I want to print a household card with a QR code, so that collection at the desk is a quick and reliable scan	Should	New in v1.1.0	S5b

4.4 Acceptance criteria

The Task 1 criteria still hold and are tested. US-09’s single criterion has been replaced by AC-09.1 to AC-09.7, which cover the invalid and corrupted scans asked about in the feedback.

Table 11. Acceptance criteria added or changed since Task 1

ID	Given / when / then
AC-09.1	Given an open event and an active household that has not collected, when a volunteer types or scans its code and presses Record collection, then the collection is stored with a timestamp, stock is deducted first-expiry-first-out, and the desk says which parcel size to hand over.
AC-09.2	Given a household that has already collected at this event, when its code is entered, then the desk says “Already collected at” the time, and offers no Record button. Only a site coordinator can record a second parcel, with a reason.
AC-09.3	Given a damaged card (part of the payload lost, or a wrong check character), when it is scanned, then the desk says the card could not be read correctly and suggests scanning again or typing the code, and nothing is sent to the server.

ID	Given / when / then
AC-09.4	Given a QR code that is not a SizaniConnect card (a web link, a payment code, a product barcode), when it is scanned, then the desk says so and asks for the household card, and nothing is sent.
AC-09.5	Given input longer than 64 characters (a scanner stuck on repeat, a pasted paragraph), then the desk says it is far too long to be a card, and nothing is sent.
AC-09.6	Given a half-typed code, then the desk says it is not a complete code and shows the format (SZN-2026-7K2QX9), and nothing is sent. After two failed scans in a row the desk suggests typing the code printed under the QR code.
AC-09.7	Given a phone with a 360-pixel screen and a slow processor (simulated at six times slower), when a printed card is held to the camera, then it is read within 5 seconds. If the camera is refused, missing or busy, a plain message says so and typing the code still works; scanning stops after 20 seconds without a read.
US-01	Given a completed form with head of household, a valid ID number, a valid South African cellphone number if one is given, a suburb and at least one member, when submitted, then the household is created as Pending Verification and its reference code is shown.
US-03	Given a pending household registered by someone else, when an operations manager verifies it, then it becomes Active and the audit log records who did it. A manager cannot verify a household they registered themselves.
US-10	Given a household with a collection at this event, when another is attempted, then it is refused with a conflict unless a site coordinator overrides it with a reason, which is stored with the collection.
US-21	Given a signed-in user, when they create recovery codes, then they are asked for their password again, ten codes are shown once, each code works once only, and creating a new set cancels the old one. A wrong code counts towards the normal lock-out.
US-22	Given an active user, when an administrator offboards them with a reason, then their next request is refused, their sessions, reset links and recovery codes are cancelled, and their future shift places are freed. An administrator cannot offboard themselves or the last active administrator.
US-23	Given an offboarded user, when an administrator erases them and types their email to confirm, then an account with no history is deleted and one with history keeps only an anonymous “Former user” row. Given households archived for more than 24 months, when the administrator anonymises them, then names, ID hashes, cellphone numbers and addresses are removed and the action is audited.
US-24	Given an active household, when its card is printed, then the card is bank-card size and shows the QR code and the reference code in large type, and scanning it at the desk finds the household (AC-09.1).

The remaining Task 1 criteria (US-02, US-11, US-12 without the attachment, US-13 and US-19) are unchanged. Every criterion in the table above is checked by an automated test (Section 10).

4.5 Business rules

Table 12. Business rules and where they are enforced

ID	Rule	Where it is enforced
BR-01	One live registration per person across all sites	Keyed hash of the ID number and a partial unique index on households

ID	Rule	Where it is enforced
BR-02	Only an Active household may collect	`isEligibleForParcel` in the domain layer
BR-03	An operations manager verifies a household before it is Active, never the person who registered it	Role policy plus a four-eyes check in `HouseholdService`
BR-04	One parcel per household per event; an override needs a coordinator and a reason	Partial unique index on collections (event, household), so it holds when two phones submit at once
BR-05	Parcel size by household size: 1 to 2 small, 3 to 5 medium, 6 or more large	Strategy classes in `domain/parcel.ts`
BR-06	First expiry first out; expired stock is never issued	`allocateFefo` with row locks on the batches
BR-07	An event opens only once enough volunteers have checked in	`EventService.open`
BR-08	A household with no collection for 12 months is archived	Archive inactive households button (v1.1.0) and `POST /households/archive-inactive`
BR-09	Every change to personal data is audited	Append-only `audit_log`, protected by a database trigger
BR-10	Check-in opens 60 minutes before a shift	`ShiftService.checkIn`
BR-11	Cellphone numbers are South African mobile numbers, stored as +27 followed by nine digits (new)	Parser in browser and API; CHECK constraints (migration 002)
BR-12	A person who leaves is offboarded first and erased only on a separate, confirmed step; history that must stay points at an anonymous row (new)	`RetentionService`; CHECK constraints (migration 004)
BR-13	An archived household’s personal data is removed 24 months after archiving (new, NFR-16)	Data retention screen; `anonymiseArchivedBefore`

4.6 User experience journey map

The journey map shows every screen and the paths between them by role. Screens added in Task 3 are S2b (recovery code), S5b (household card), S23 (data retention) and S24 (my account); S2c (choose a password) existed in Task 2 for activation and reset links but was not shown on the Task 1 map.

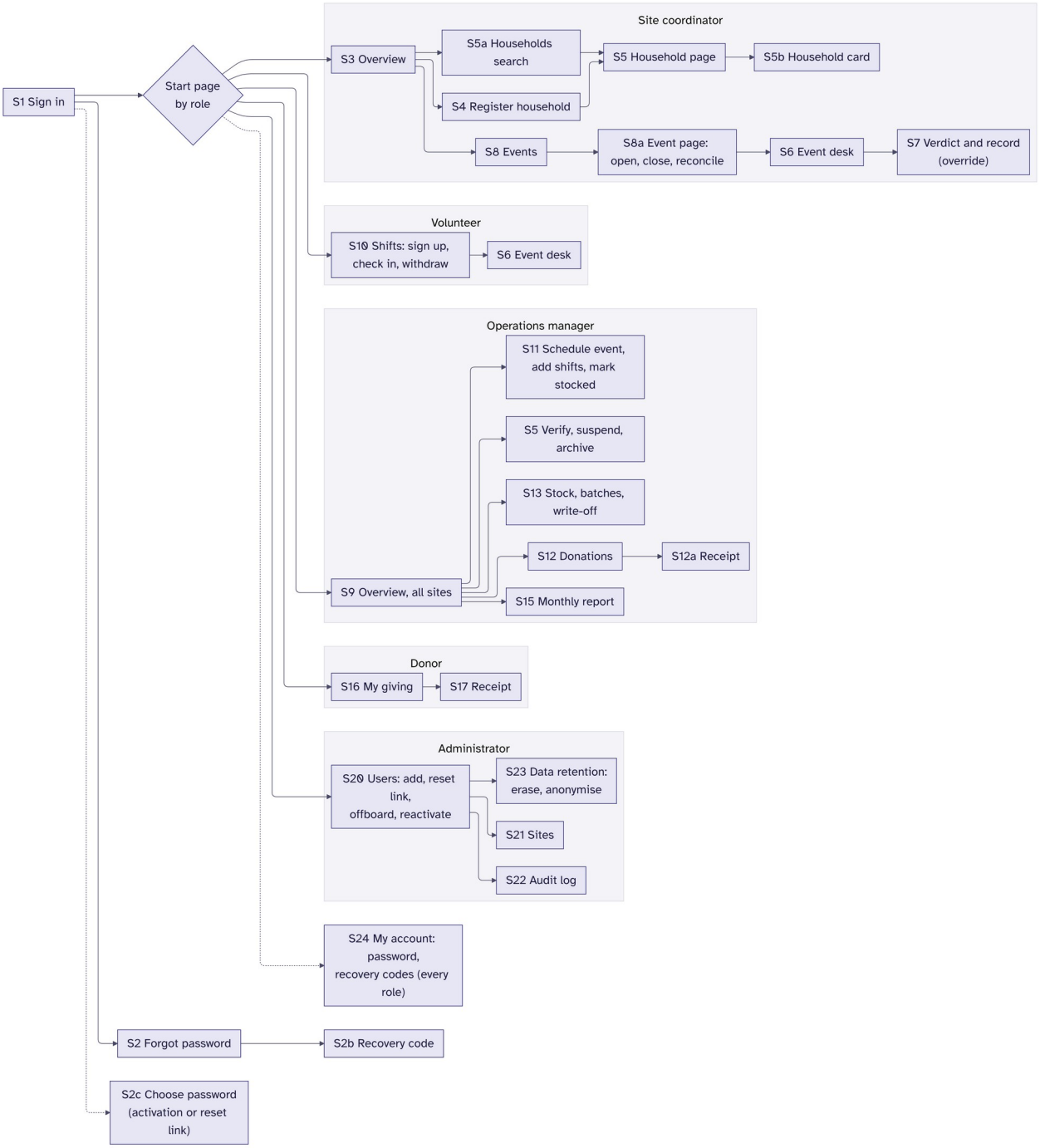


Figure 1. User experience journey map, by role (v1.1.0)

4.7 Non-functional requirements: how each was verified

Task 1 set a measurable target for every non-functional requirement so that it could be tested. The table below gives the result for each. Three are not fully verified, and the table says so rather than claiming them.

Table 13. Non-functional requirements and their verification

ID	Target (short)	Result	Evidence
NFR-01	Search p95 under 2 s with 5,000 households	**Met.** p95 0.43 s (name search), 0.09 s (desk lookup)	Load test, Section 10.3
NFR-02	Record a collection p95 under 1.5 s	**Met.** p95 0.65 s with ten desks at once	Load test
NFR-03	Monthly report under 8 s, export under 20 s	**Met.** p95 0.09 s and 0.07 s	Load test
NFR-04	99.5% availability a month	**Designed for, not yet measured over a month.** Health check with automatic restart; specific failure answers	Section 11.5
NFR-05	10,000 households, 150 concurrent users, scale by instance count	**Partly verified.** Stateless API (scale-out needs no change); tested at 5,050 households	Load test; Section 9.3
NFR-06	No confirmed collection lost; multi-table writes in one transaction	**Met.** Unit of Work; race test; failure tests show nothing half-saved	`operations.test.ts`, `failures.test.ts`
NFR-07	7-day backups, RPO 24 h, RTO 4 h	**Met by configuration.** 7-day backups with point-in-time restore; procedure documented (about an hour), not rehearsed on production	Section 11.4
NFR-08	Untrained volunteer records a collection within 3 minutes (5 participants)	**Not yet measured.** Needs a session with real volunteers; the desk is one field and one button	Section 14
NFR-09	Confirm destructive actions; errors say what to do next	**Met.** Confirmation before archive, offboard, erase and anonymise; error messages say what to do next	`admin.spec.ts`; user guide 9.1
NFR-10	WCAG 2.1 AA	**Met for automated checks.** axe-core on every main screen for every role at 1280 and 360 px; no manual screen-reader test yet	`accessibility.spec.ts`
NFR-11	Chrome, Edge, Firefox, Safari; Android at 360 px	**Partly verified.** Automated tests run in Chromium, the engine of Chrome and Edge; Firefox and Safari are not yet tested	Section 10
NFR-12	360, 768, 1280 px; no sideways scrolling; 44 px touch targets	**Met.** No sideways scrolling checked at 360 px on every screen; 44 px minimum on buttons and inputs	`accessibility.spec.ts`
NFR-13	HTTPS with TLS 1.2+; database not reachable from the internet	**Met with a deviation.** TLS 1.2+ and HSTS. The database has a public endpoint but its firewall admits only the web apps, over TLS	Sections 3.2, 9.3

ID	Target (short)	Result	Evidence
NFR-14	PBKDF2 at least 100,000 iterations; no credential or identifier in a log	**Met.** PBKDF2-SHA256 600,000 iterations; logs redact secrets and, since v1.1.0, keep no query strings	`security.test.ts`, `logging.test.ts`
NFR-15	ID numbers stored only as a hash; minimal data	**Met, stronger than planned.** HMAC-SHA256 with a key outside the database; age bands instead of birth dates	`households.test.ts`
NFR-16	Anonymise archived households after 24 months	**Met (new in v1.1.0).** Administrator runs it from the Data retention screen, which lists what is due	`retention.test.ts`
NFR-17	Audit every change, append-only	**Met.** Trigger refuses UPDATE and DELETE on `audit_log`	`households.test.ts`
NFR-18	Test coverage at least 70%, enforced in CI	**Met.** API statements 96.9%, branches 86.7%; CI fails below 70%	Section 10.2
NFR-19	No vendor-specific service in business logic; runs in a container on any PostgreSQL	**Met.** Dockerfile and Docker Compose; plain PostgreSQL 16	User guide A.2
NFR-20	dd MMM yyyy dates, Rand, South African English	**Met.** en-ZA formatting; cellphone numbers shown as 082 555 0134	`format.ts`

5. Project plan for Task 3

Task 1's work breakdown structure covered Task 1 only. The plan below covers Task 3, responding to feedback F2. It has one resource, Byron Bousfield, because Group 13 has one member, so the plan is levelled: tasks run one after another, never in parallel. Contention is planned for rather than discovered: wherever two tasks would have run side by side in a larger team, the plan names the conflict and the rule used to resolve it.

5.1 Work breakdown structure

Table 14. Task 3 work breakdown

#	Task	Duration	Depends on	Status at 29 Oct
1	Read the feedback; turn each point into a change and a test	1 day	RC-1	Done
2	Dependency updates; migration rollback tooling (F5)	1 day	1	Done
3	Cellphone validation, migration 002 (F6)	1 day	2	Done
4	Administrator self-recovery, migration 003 (F7)	1 day	2	Done
5	Offboarding, erasure and retention, migration 004 (F8)	2 days	2, 4	Done
6	Household cards and scan handling (F1)	1 day	1	Done
7	Failure handling and runbook (F3, F4)	1 day	2	Done
RC-2	Code freeze: no new features after this point	—	2 to 7	Reached
8	End-to-end and accessibility tests in CI	1 day	RC-2	Done
9	Load test for NFR-01 to NFR-03	1 day	8	Done
10	User guide with annotated screenshots	2 days	8	Done
11	Walkthrough video	1 day	10	Done
12	Final report and diagrams	2 days	9, 10	Done
RC-3	Release v1.1.0 to production (approval in GitHub)	—	8, 9	Ready, awaiting approval
13	Hand in the project report	30 Oct	12, RC-3	Due
14	EXPO PowerPoint and display board	2 days	12	Done
15	Print the board and cards; set up an open event for the day	2 days	14	Planned, 4 to 5 Nov
16	Rehearse the demonstration; buffer for slips	3 days	15	Planned, 6 to 9 Nov
17	EXPO	10 Nov	16	Planned

5.2 Contention and how it was resolved

Table 15. Resource contention for a one-person team

Where tasks compete	Rule applied
RC-1: the Task 2 presentation (14 Oct) against the start of Task 3	Task 3 started only after the presentation was given, and only from the released v1.0.1, so nothing presented was changed underneath the presentation.
Tasks 3 to 7 all change the database schema	Run strictly one after another, one migration each, in the order they were numbered, so each migration and its down script is written against a known schema. Parallel branches would have produced migrations that conflict.
Tasks 8 to 11 could have run beside the feature work	Held until RC-2. Screenshots, the video and accessibility checks describe the screens, so they wait for the screens to stop changing. Doing them earlier would mean doing them twice.
The report (12) and the EXPO material (14)	The report comes first because it is due on 30 October; the EXPO follows on 10 November. The deck and board reuse the report's diagrams and the guide's screenshots.
Other modules' deadlines before the EXPO: PROG7314 on 6 November and INSY7314 on 9 November	The EXPO deck and display board were finished with this report on 29 October, so the weeks of those deadlines hold only printing, setting up the demonstration event and one rehearsal, with a day of buffer before 10 November.

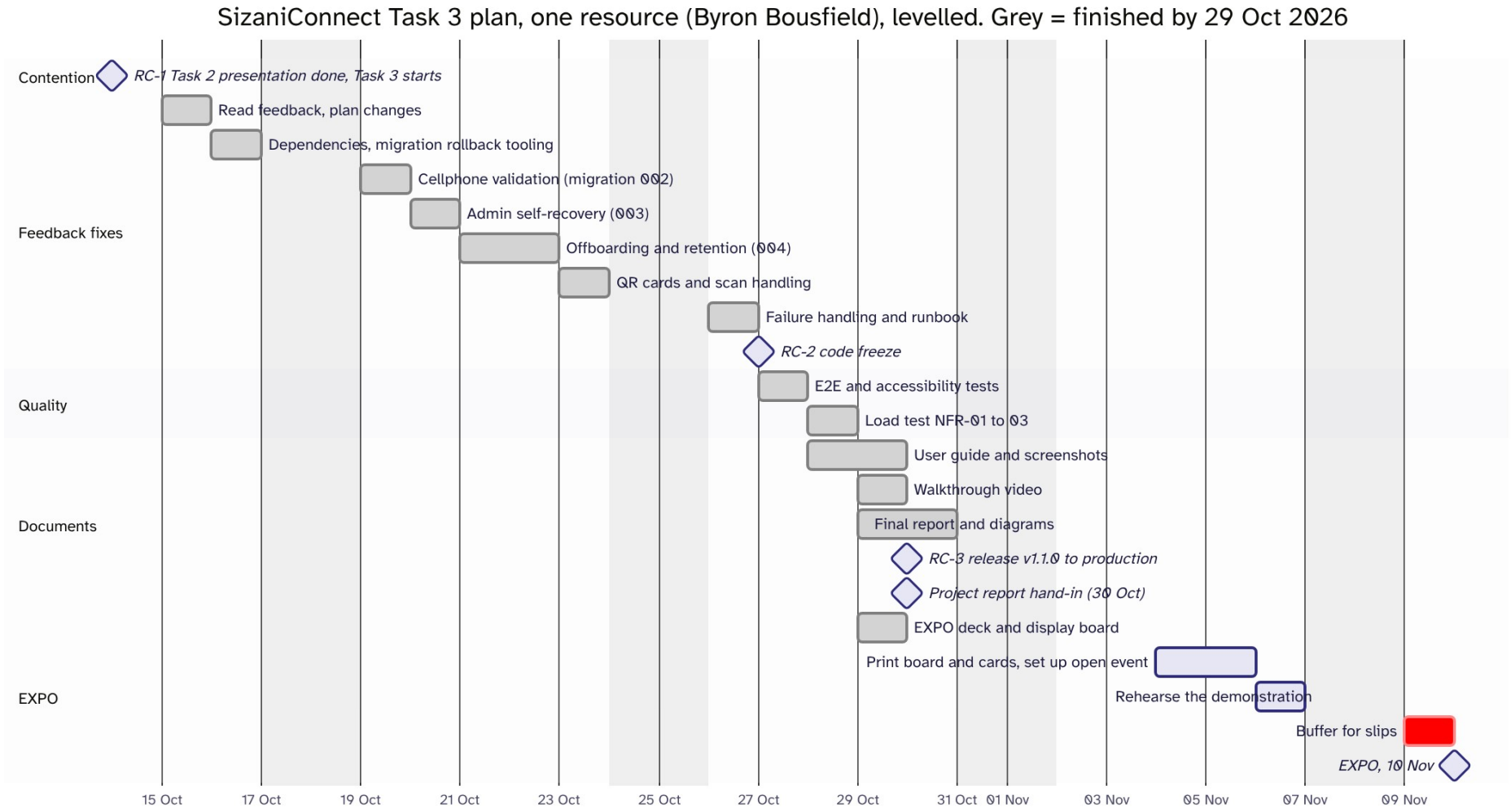


Figure 2. Task 3 plan, levelled for one resource, with release-candidate milestones

6. Analysis and design

6.1 Domain model

The domain still divides into the three clusters identified in Task 1: who is served (Household, HouseholdMember), how service is delivered (DistributionSite, DistributionEvent, Collection, VolunteerShift, ShiftAssignment) and what is delivered and where it came from (Donor, Donation, StockItem, StockBatch, StockMovement). User links to all three. Task 3 adds RecoveryCode to the user, and lifecycle attributes (offboarded, erased, archived, anonymised) to User and Household.

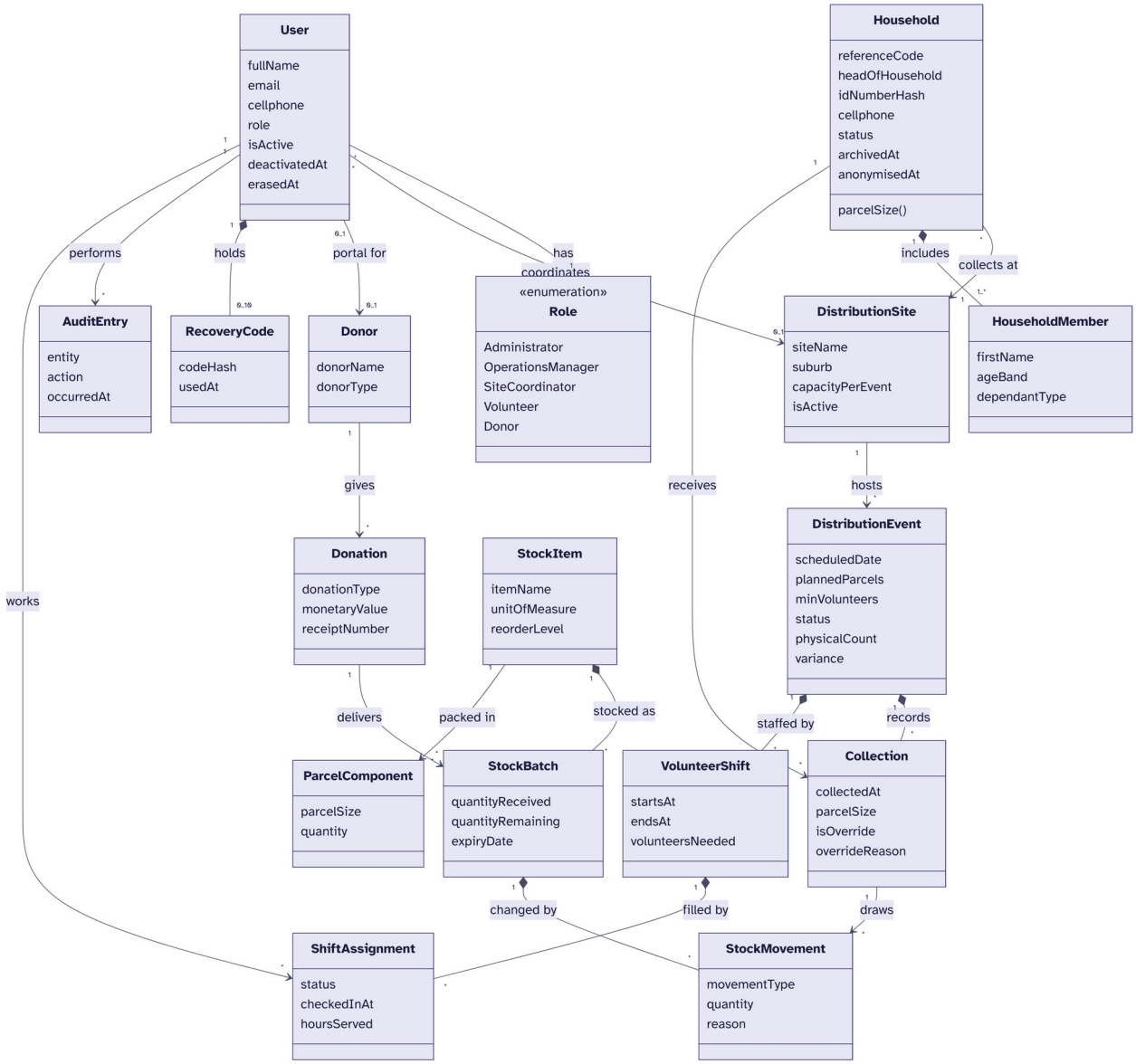


Figure 3. Domain model (v1.1.0)

6.2 Design class diagram

As in Task 1, the household registration slice is shown in full; the other slices have the same shape. The route calls a service, the service receives everything it needs through one injected dependency object, and all database access happens inside a unit of work that hands out repositories bound to

one transaction. Business rules live in small domain modules with no input or output, so they are tested without a database.

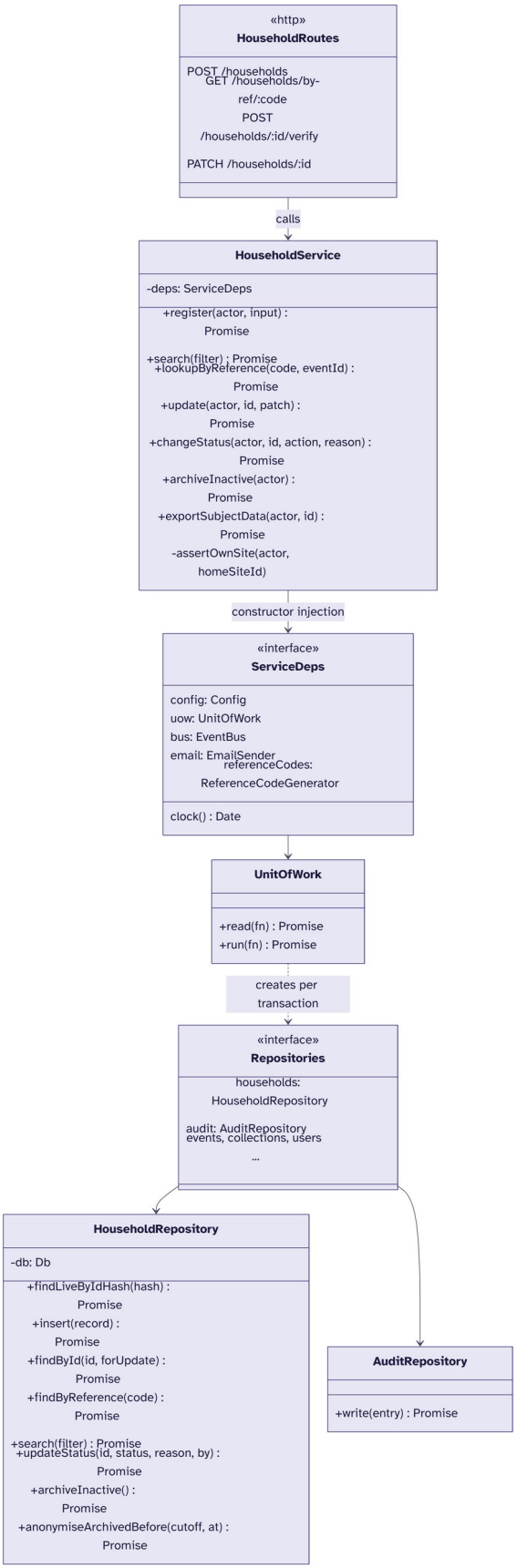


Figure 4. Design classes, household registration slice: from the HTTP route to the database

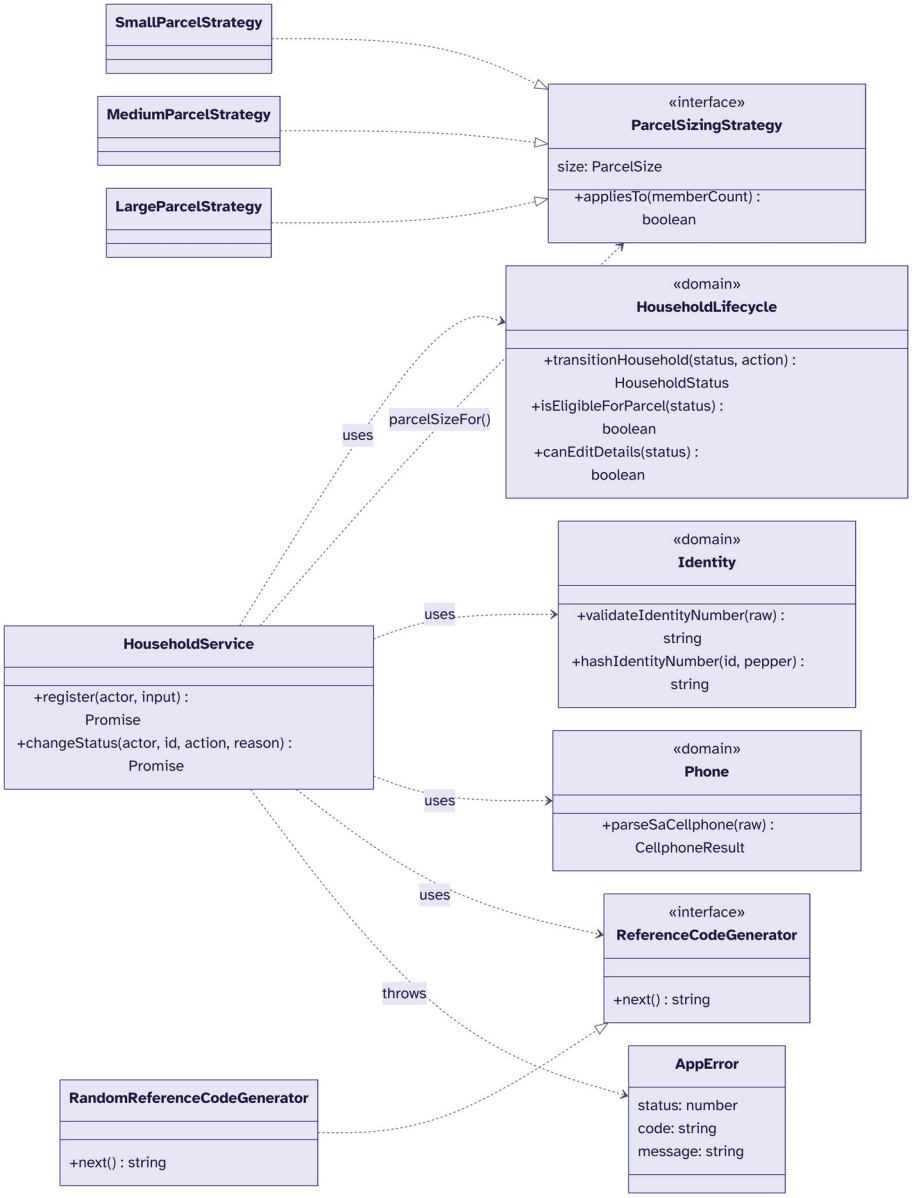


Figure 5. Design classes: the domain rules and strategies the household service uses

Table 16. Classes added in Task 3

Class added in Task 3	Responsibility
`RetentionService`	Offboarding, erasure and household anonymisation (F8, NFR-16)
`RecoveryCodeRepository`, `security/recoveryCodes.ts`	Creating, hashing and using single-use recovery codes (F7)
`domain/phone.ts` (and `web/src/lib/phone.ts`)	Parsing and formatting South African cellphone numbers (F6)
`parseScan`, `cardPayload` in `domain/referenceCode.ts`	Building and checking household card payloads (F1)
`db/failures.ts`	Turning database failures into specific 503 answers (F4)
`db/cli.ts`, `db/migrate.ts`	Migration status, rollback and break-glass recovery (F5, F7)

7. Behaviour: sequence and state diagrams

7.1 Sequence diagrams

Task 1 documented three flows. The final system has six, because the feedback asked for alternative and failure paths, and because account recovery and offboarding are new flows that cross the system boundary. Every diagram uses the real class, route and error names from the code, and each alternative fragment (alt) is a path that an automated test exercises.

Table 17. Sequence diagrams

Diagram	Flow	Alternatives shown
SD-01	Register and verify a household	Same person already registered (BR-01); two registrations at the same instant; four-eyes check on verification
SD-02	Record a collection at the event desk	Bad scan refused in the browser (AC-09.3 to 09.6); not eligible; already collected; coordinator override; first-expiry-first-out stock deduction
SD-02b	The same collection when the network or database fails (new, F3)	TLS handshake times out; saved but the answer is lost; database unreachable
SD-03	Capture an in-kind donation	Validation failures; one stock batch per line; receipt number from a sequence
SD-04	An administrator signs in with a recovery code (new, F7)	Unknown email or locked account given the same answer as a wrong code; wrong code counts towards lock-out
SD-05	Offboard and later erase a volunteer who retires (new, F8)	Self or last administrator refused; deleted outright or anonymised, depending on history

The diagrams follow on landscape pages so they can be printed at a readable size.

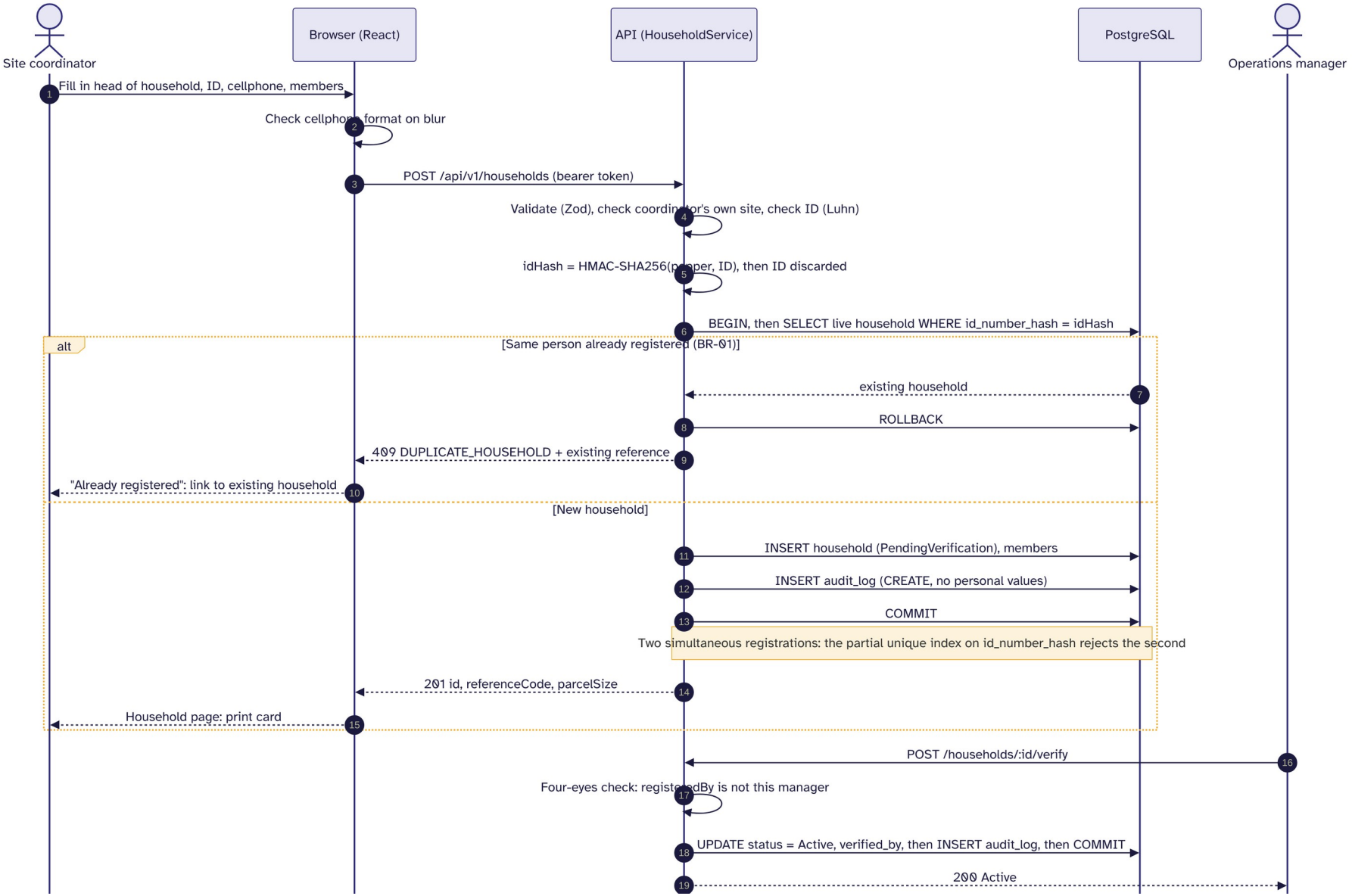


Figure 6. SD-01: register and verify a household

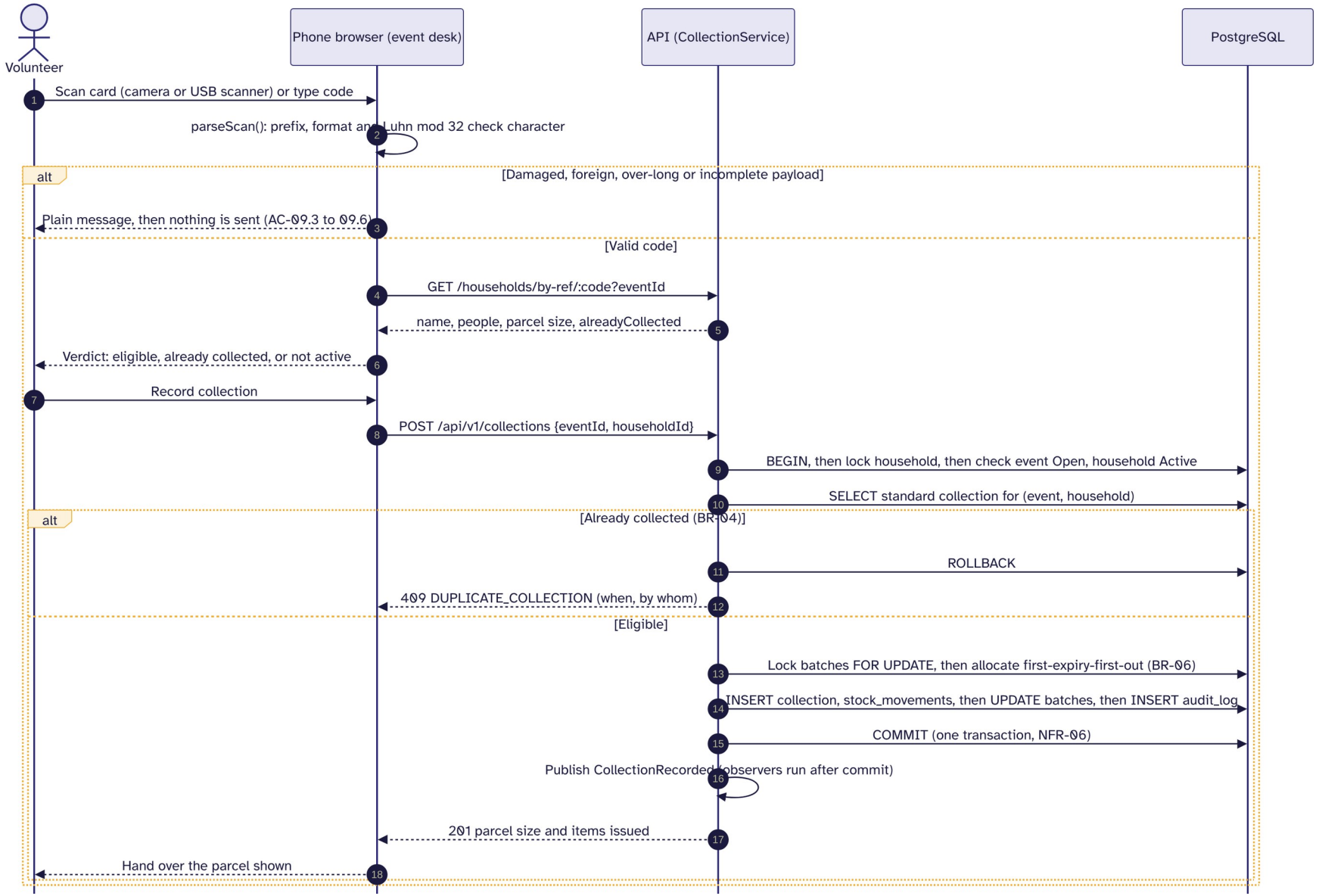


Figure 7. SD-02: record a collection at the event desk, including bad scans and the duplicate path

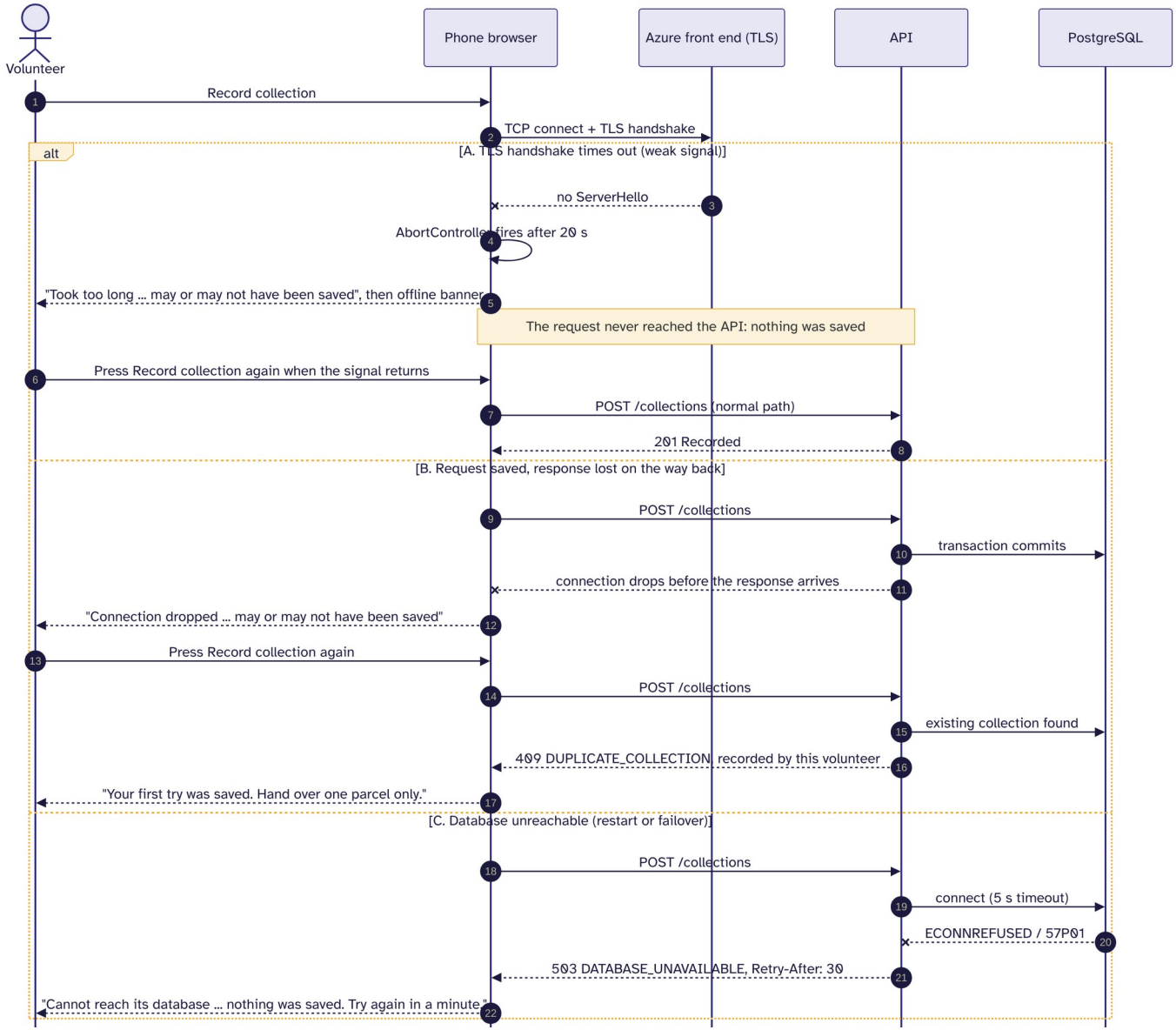


Figure 8. SD-02b: the same collection when the TLS handshake times out, the answer is lost, or the database is unreachable (feedback F3)

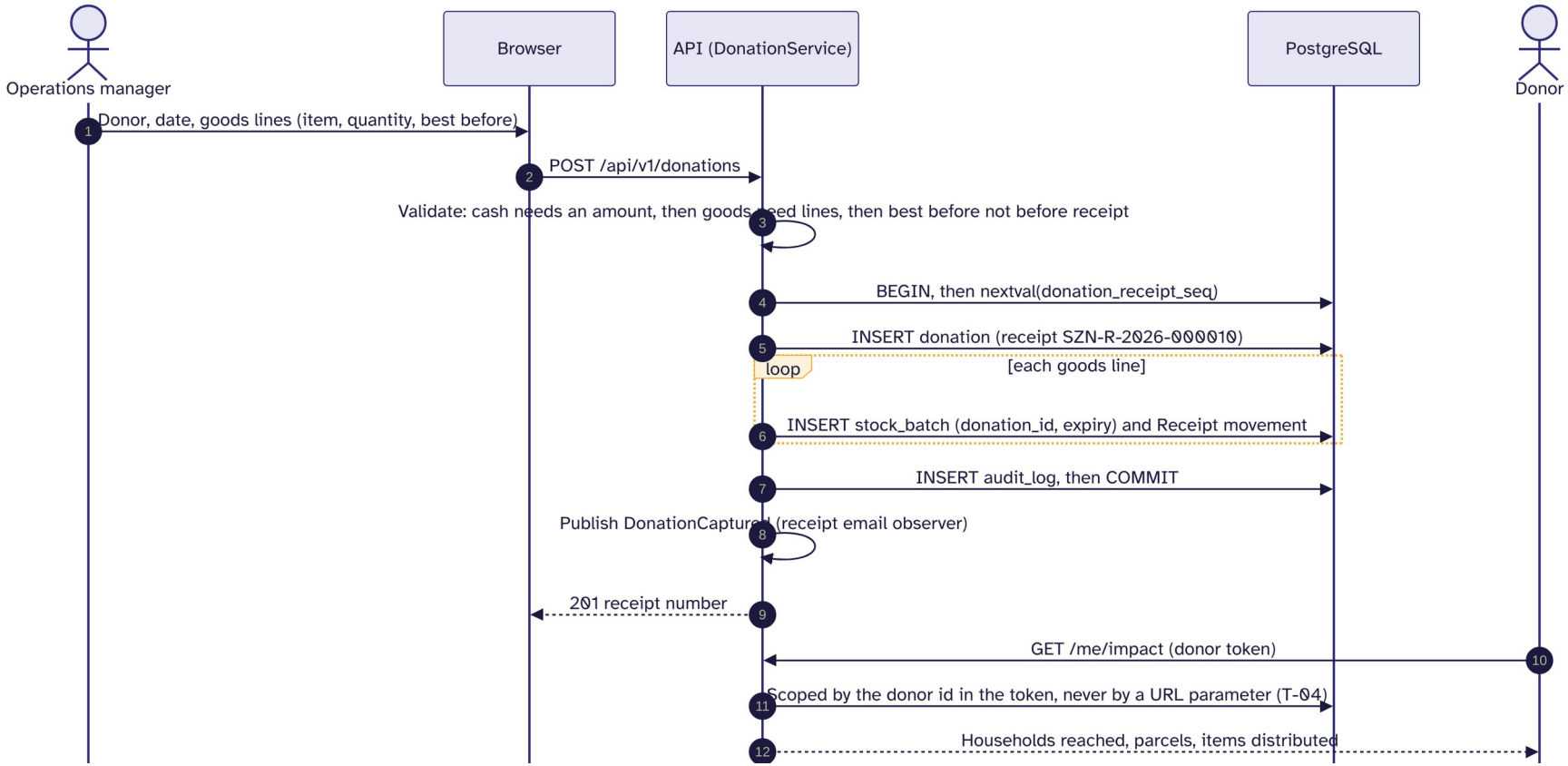


Figure 9. SD-03: capture an in-kind donation and create stock batches

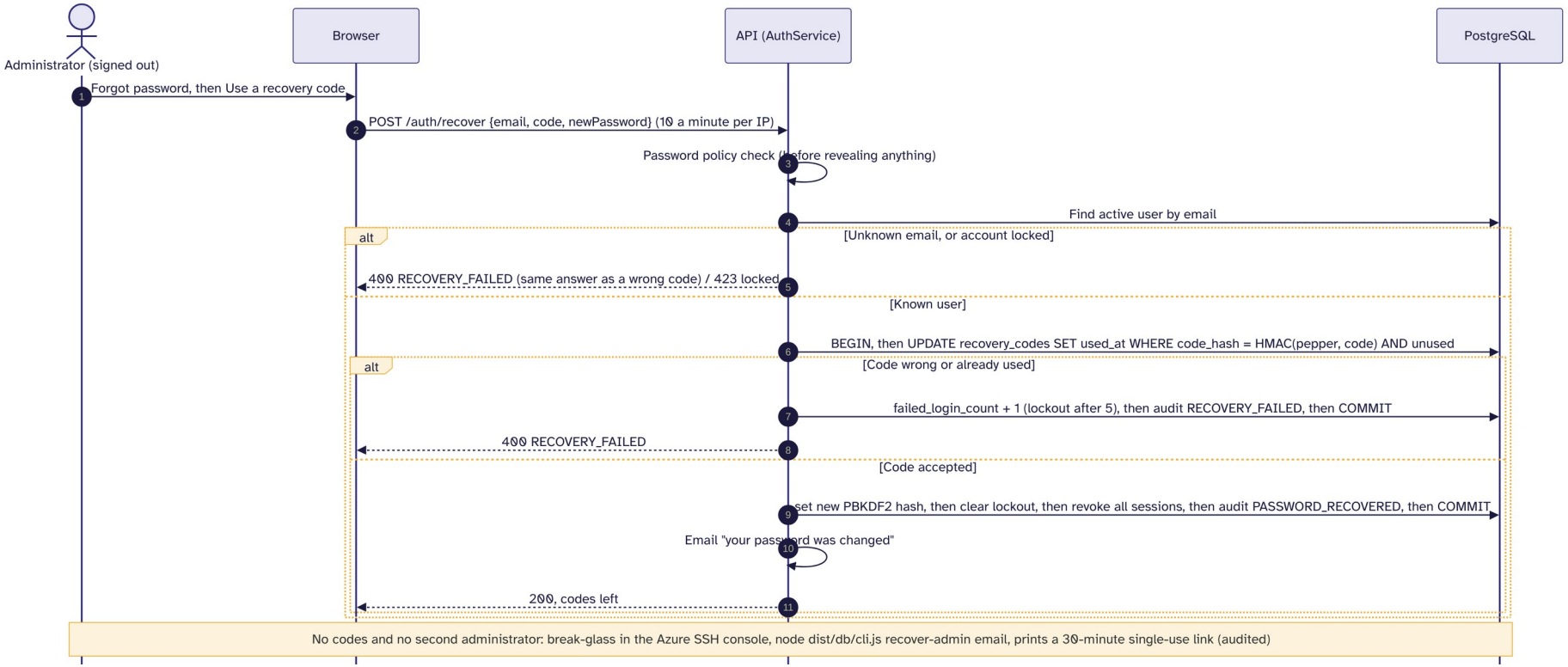


Figure 10. SD-04: an administrator who forgot the password signs in with a recovery code (feedback F7)

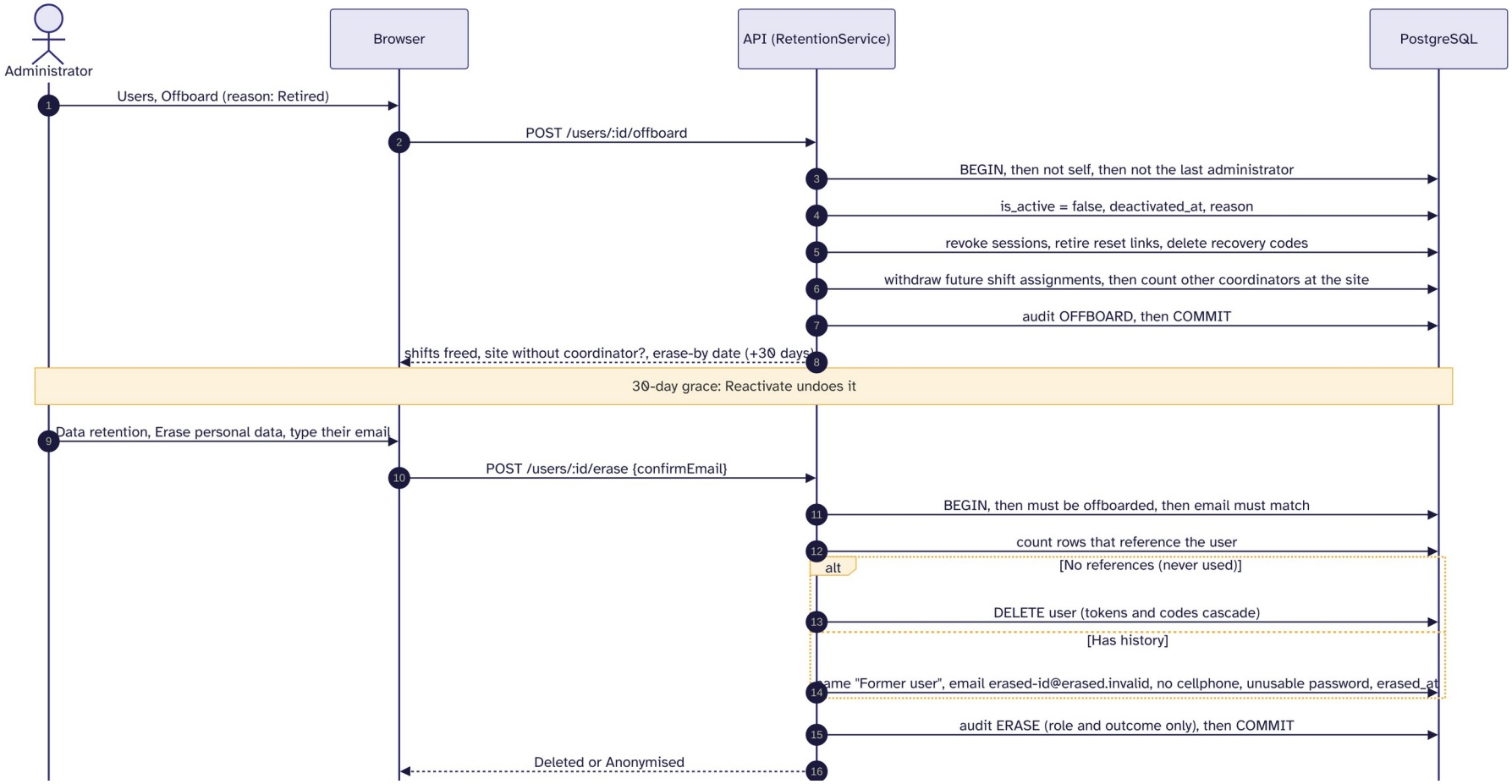


Figure 11. SD-05: offboard a volunteer who retires, then erase their personal data (feedback F8)

7.2 State diagrams

Task 1 had two state diagrams, for the household and the distribution event. Both still match the code exactly: the allowed transitions are a table in `domain/household.ts` and `domain/distributionEvent.ts`, and anything else is refused with INVALID_STATUS_CHANGE. Three diagrams are new. The user lifecycle shows offboarding and erasure (F8). The event desk shows how a scan is handled, including bad scans and an uncertain save (F1, F3). The system availability diagram shows the failure states and how the system recovers from each (F4). Transitions are labelled with short codes so the diagrams stay legible; each table explains its codes.

Household lifecycle

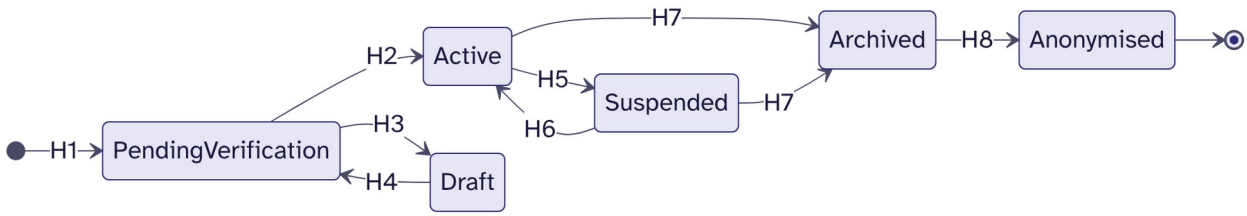


Figure 12. Household lifecycle

Table 18. Household transitions

Code	Transition and guard
H1	A coordinator registers the household (US-01). Refused if the same person already has a live registration anywhere (BR-01).
H2	An operations manager verifies it (US-03): never the person who registered it (BR-03).
H3	Returned for correction with a reason.
H4	Corrected and resubmitted by a coordinator at its own site.
H5	Suspended with a reason. A suspended household cannot collect (BR-02).
H6	Reinstated.
H7	Archived by a manager, or by Archive inactive households after 12 months without a collection (BR-08). An archived household must register again.
H8	Anonymised by an administrator 24 months after archiving (NFR-16, BR-13): name, ID hash, cellphone and address removed; counts kept. The status stays Archived with an anonymised date, so this is a sub-state of Archived.

Distribution event lifecycle

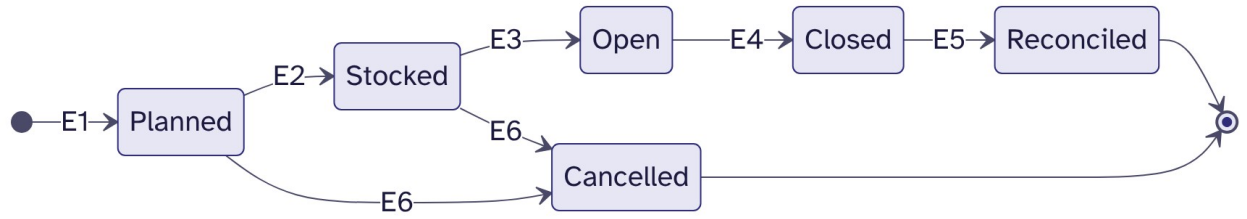


Figure 13. Distribution event lifecycle

Table 19. Event transitions

Code	Transition and guard
E1	Scheduled by an operations manager for a site, date and time (US-05).
E2	Marked as stocked once parcels are packed.
E3	Opened by the site's coordinator, only once enough volunteers have checked in (BR-07). Collections can be recorded only while Open.
E4	Closed at the end of the distribution.
E5	Reconciled: the physical count is entered and any difference from the system's count is explained in at least ten characters (US-11).
E6	Cancelled, only before it opens.

User lifecycle (new, feedback F8)

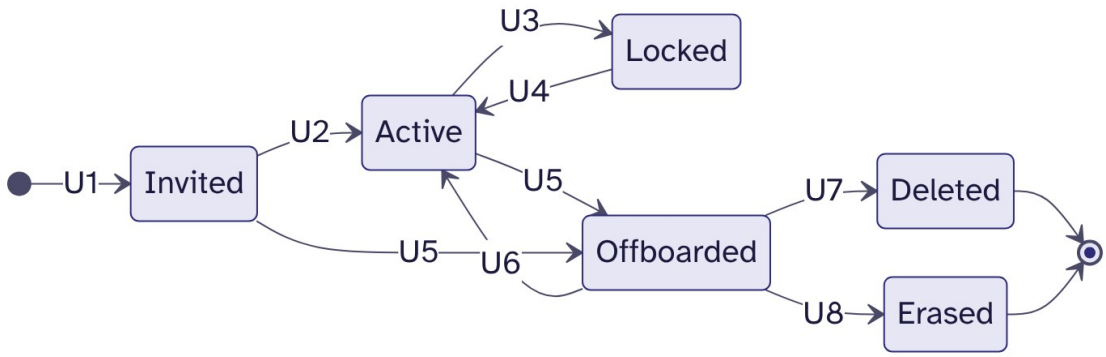


Figure 14. User account lifecycle, including offboarding and erasure

Table 20. User transitions

Code	Transition and guard
U1	An administrator creates the account (US-18). It has no password yet; an activation link is emailed or shown to the administrator.
U2	The person chooses a password through the link.
U3	Locked after five failed sign-ins within 15 minutes (T-01). Wrong recovery codes count too.
U4	Unlocks by itself after 15 minutes, or at once through a password reset.
U5	Offboarded by an administrator with a reason (Retired, Resigned, Role ended, Other). Sessions, reset links and recovery codes are cancelled and future shift places freed. Not allowed for one's own account or the last active administrator.
U6	Reactivated by an administrator, possible until the account is erased. Erasure becomes due 30 days after offboarding.
U7	Erased, never used: an account with no history is deleted outright.
U8	Erased, with history: name, email, cellphone and password are removed and the row becomes "Former user", so collections, shifts and the audit trail still point at it.

Event desk (new, feedback F1 and F3)

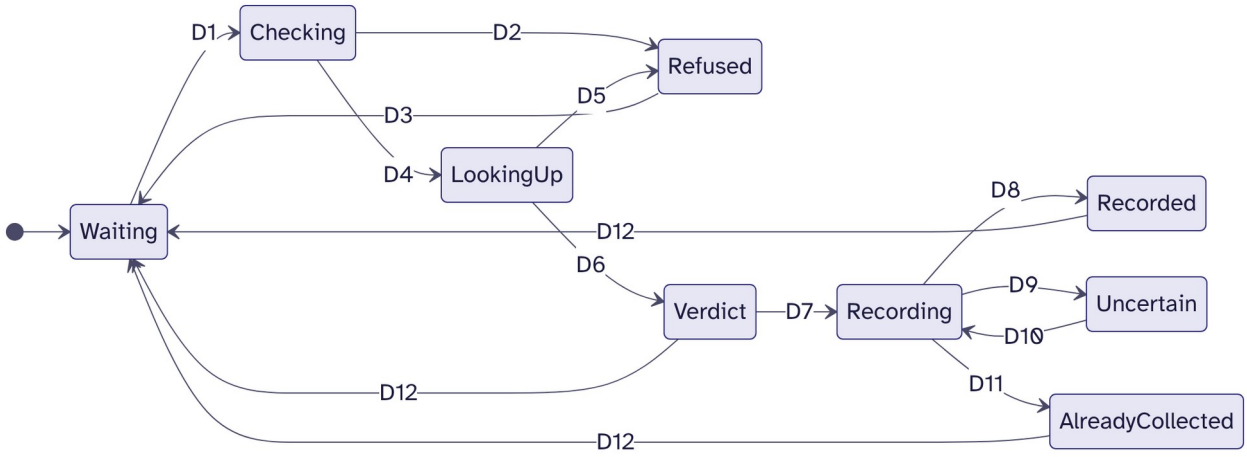


Figure 15. Event desk: handling one scan or typed code

Table 21. Event desk transitions

Code	Transition
D1	A code is typed, scanned by a USB scanner, or read by the camera.
D2	The browser's check fails: damaged, foreign, over-long or incomplete (AC-09.3 to 09.6). Nothing is sent.
D3	The volunteer scans again or types the code. After two failures the desk suggests typing it.
D4	The code is valid, so the desk looks it up: GET /households/by-ref.
D5	Not found, not eligible, or the card was refused by the server's copy of the same check.
D6	A verdict: eligible with the parcel size, or already collected (AC-09.1, AC-09.2).
D7	The volunteer presses Record collection.
D8	Saved: hand over the parcel.
D9	No answer within 20 seconds, or the connection dropped: the save may or may not have happened.
D10	The volunteer presses Record collection again when the signal returns. This is safe (BR-04).
D11	The repeat finds the first attempt was saved by this volunteer: shown as saved, hand over one parcel only.
D12	Ready for the next household.

System availability (new, feedback F4)

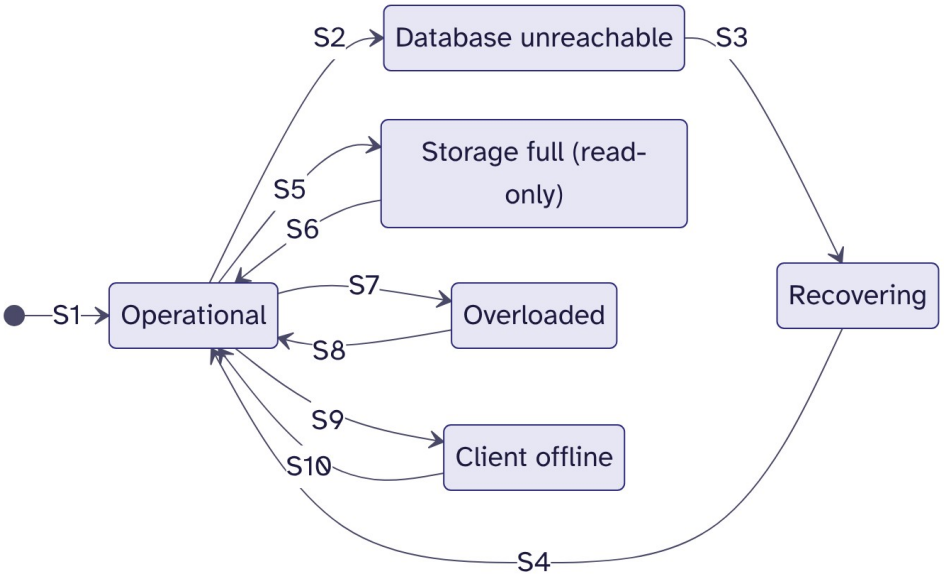


Figure 16. System availability: failure states and recovery transitions

Table 22. System availability transitions

Code	Transition	What users see and what happens
S1	Start: migrations applied under a lock; health check reports ok	Normal service
S2	The database stops answering (restart, failover, firewall change)	Every request answers 503 DATABASE_UNAVAILABLE, “nothing was saved, try again in a minute”; the health check reports down and App Service watches it
S3	The database answers again	The pool opens new connections on the next request; broken ones are discarded
S4	First successful request	Normal service, without restarting the app
S5	Storage passes 95%: Azure makes the server read-only	Pages still load; every save answers 503 STORAGE_FULL; health reports degraded. Auto-grow and an 80% alert make this unlikely
S6	Storage is raised in the portal	Saves work again within a minute; no restart
S7	Too many connections, or a statement runs longer than 15 seconds	503 DATABASE_BUSY or DATABASE_TIMEOUT with Retry-After; the write was rolled back
S8	Load drops	Normal service
S9	The phone loses its connection, or the TLS handshake never completes	The browser gives up after 20 seconds and shows a yellow banner; reads are retried once, writes never automatically
S10	Any request succeeds	The banner clears itself

8. Data schema

8.1 Technology

PostgreSQL 16 on Azure Database for PostgreSQL Flexible Server remains the data store, for the reasons given in Task 1: a strongly relational domain, transactions that span collections and stock (NFR-06), constraints the database enforces itself, and no lock-in (NFR-19). The one change is how the application reaches it. Instead of Entity Framework, the repositories use the `pg` driver with hand-written parameterised SQL, and a lint rule fails the build if a value is ever interpolated into an SQL string (T-03). The schema is versioned as numbered SQL migrations in the repository, each new one with a down script.

8.2 Entity relationship diagrams

The overview shows all 19 tables with their keys and relationships. The three diagrams after it show every column, one cluster at a time: access (people, sessions and the audit trail), distribution (households, events, collections and shifts) and supply (donors, donations and stock).

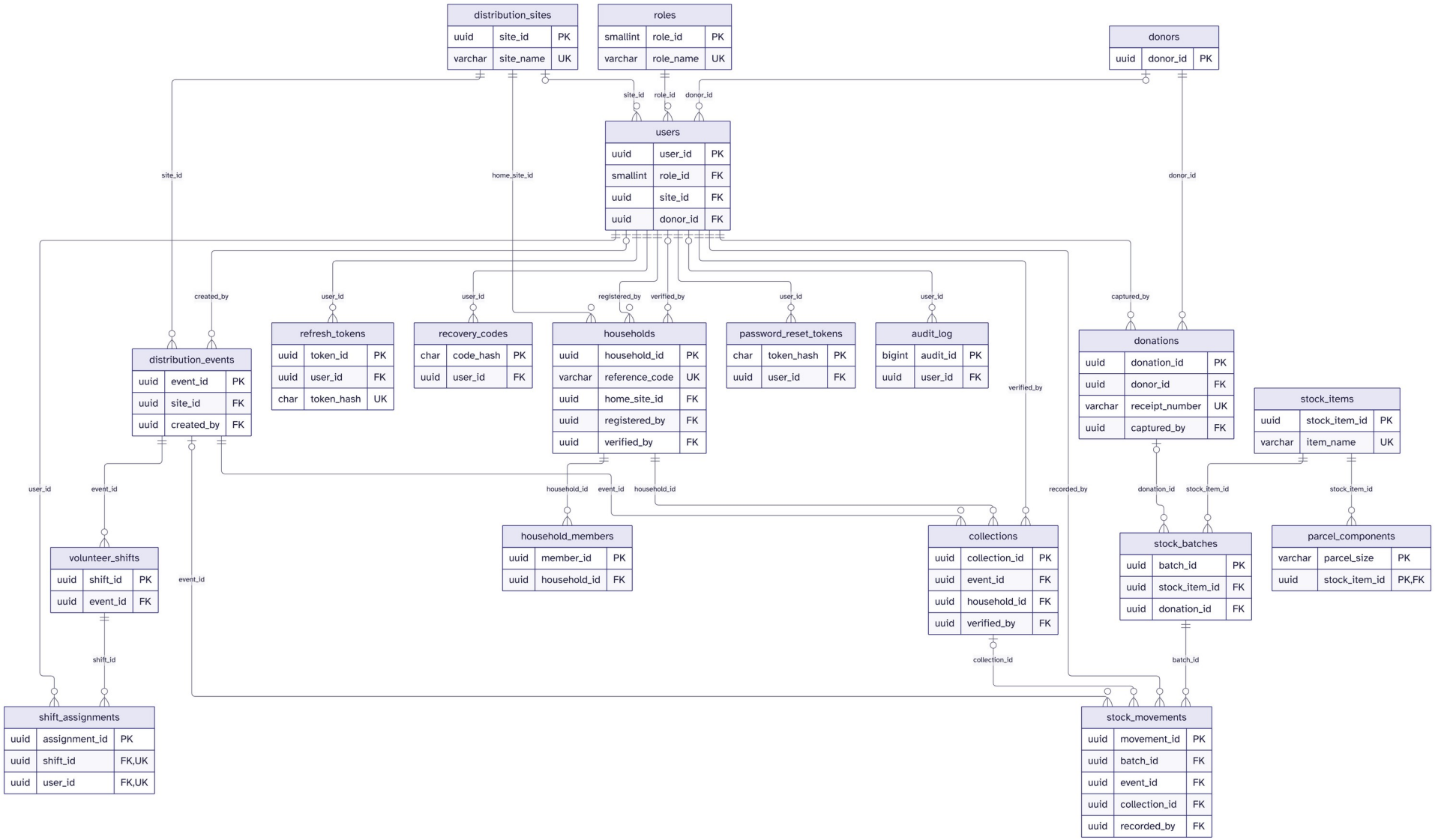


Figure 17. Entity relationship overview: all tables with primary, foreign and unique keys (v1.1.0)

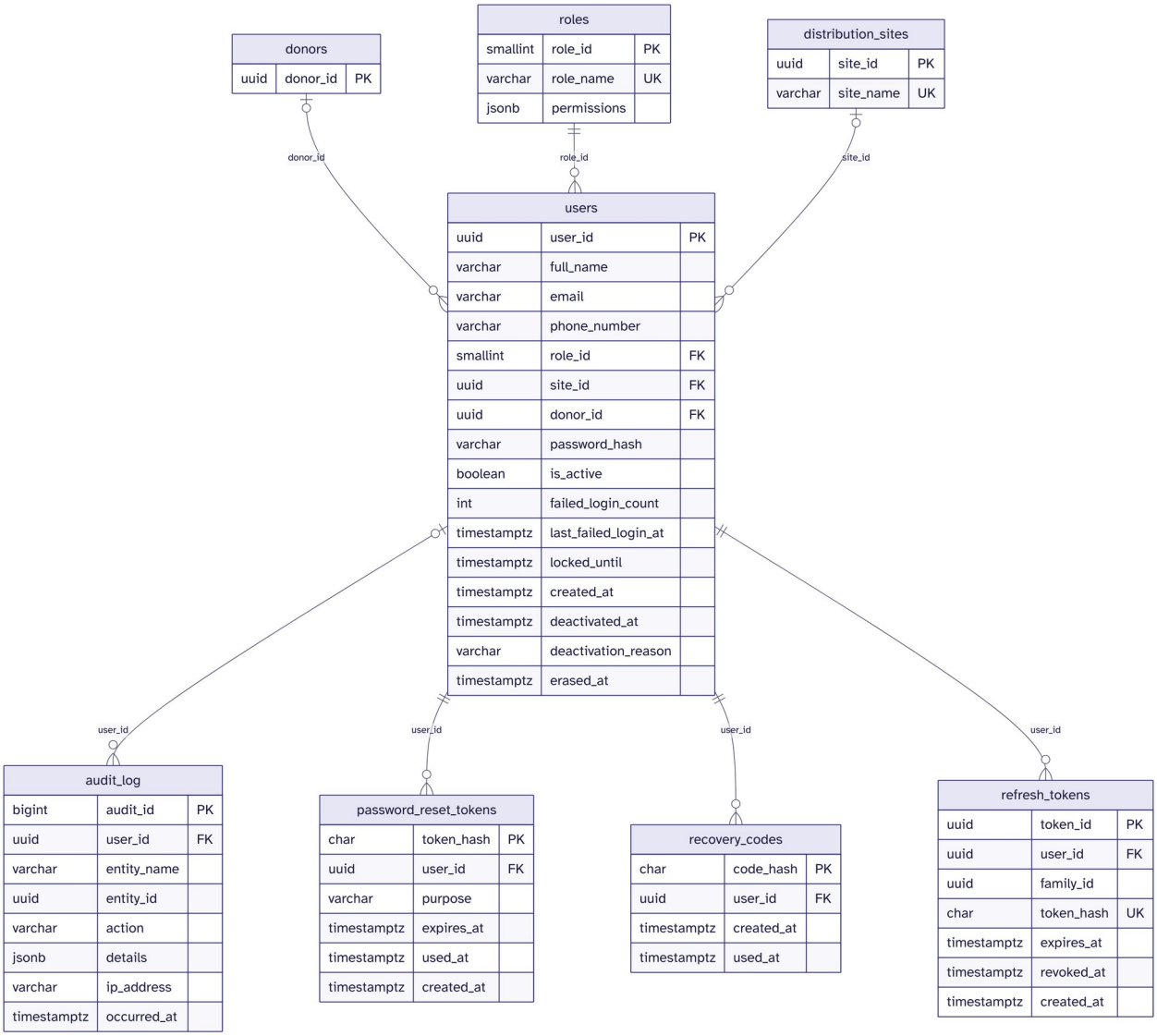


Figure 18. ERD, access cluster: users, roles, sites, sessions, reset tokens, recovery codes and the audit log

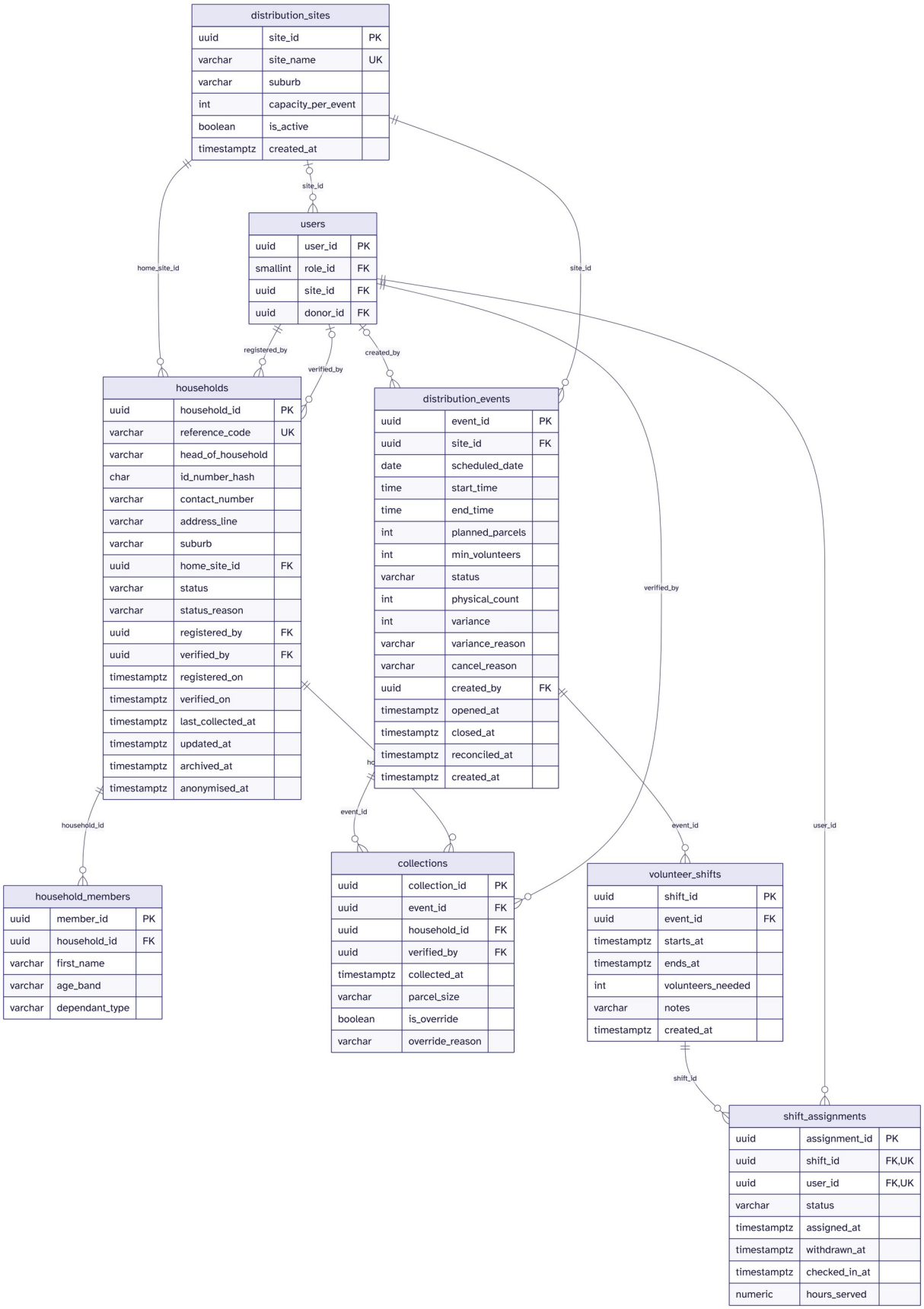


Figure 19. ERD, distribution cluster: households, members, events, collections and shifts

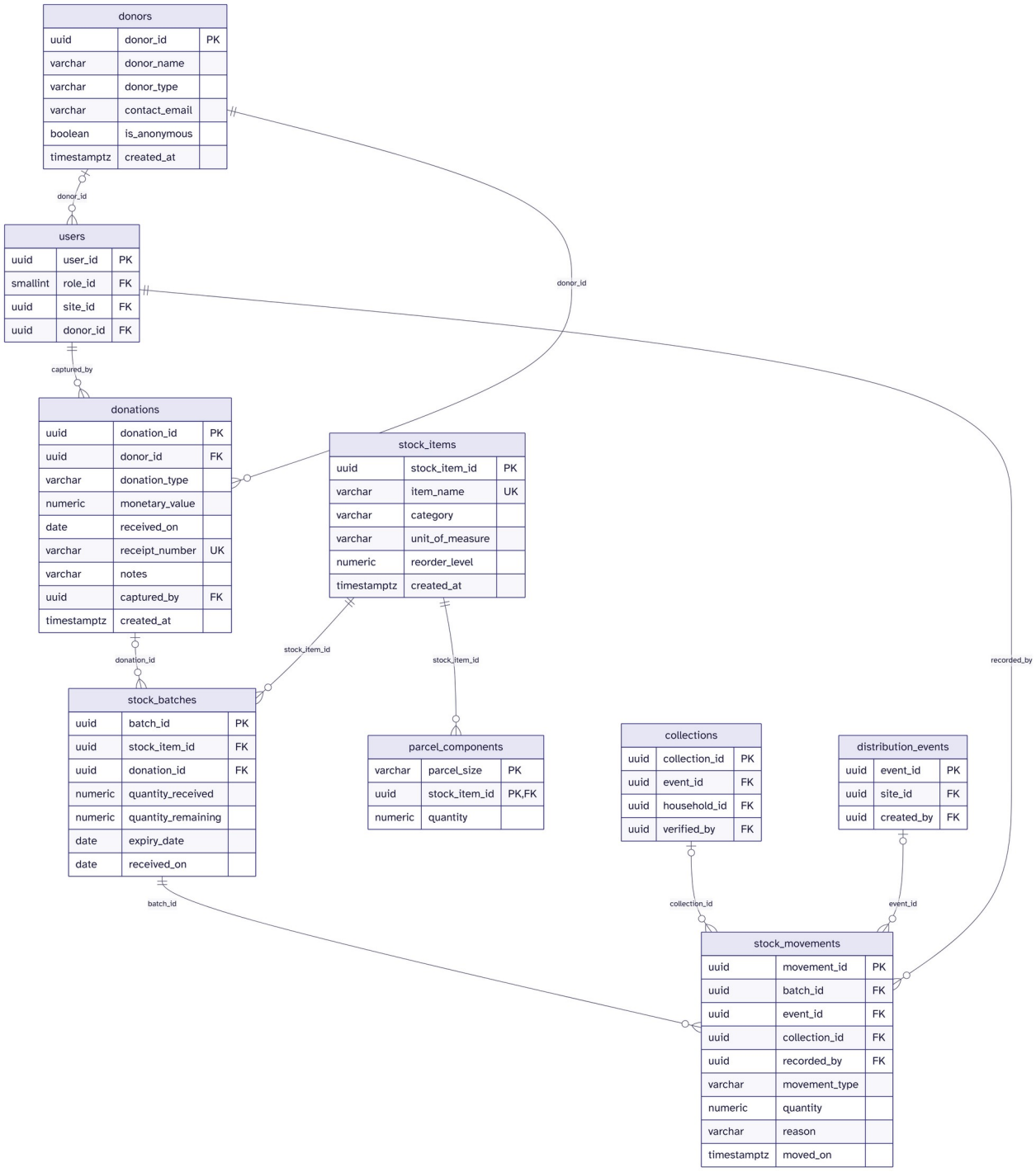


Figure 20. ERD, supply cluster: donors, donations, stock items, batches, movements and parcel contents

8.3 Schema design decisions

Table 23. Schema design decisions

Decision	Reason
UUID primary keys on business tables	Identifiers appear in addresses; sequential numbers would reveal the size of the register (T-12).
A separate random reference code	Short and readable aloud at the desk, printed with a QR code; reveals nothing.

Decision	Reason
`id_number_hash` (HMAC) instead of the ID number	NFR-15. Supports the duplicate check (BR-01) without storing the number in any recoverable form.
Partial unique index on collections (event, household) where not an override	BR-04 holds even when two phones record the same household at the same instant.
Partial unique index on live households' ID hash	BR-01 holds under concurrency; archived households do not block a new registration.
Stock batches separate from stock items, with `quantity_remaining` CHECK (>= 0)	Expiry belongs to a consignment (BR-06), and over-issuing is impossible.
`stock_movements` append-only	Every change in stock is explained by a movement linked to a collection, donation, write-off or event.
`audit_log` append-only by trigger	NFR-17 and BR-09: UPDATE and DELETE on the table raise an error. Entries record who, what and when, never personal values.
CHECK constraints for cellphone format (002)	BR-11 holds whatever writes to the table.
Lifecycle columns with CHECK constraints (004)	An offboarded user must have a date and reason; an anonymised household must be archived.
`recovery_codes` holds only keyed hashes (003)	A copy of the database does not reveal anyone's codes.
Indexes on the hot paths	Household search, desk lookup, collections by event, batches by item and expiry, events by site and date, and the retention queue.

8.4 Migrations and how each is undone

Table 24. Migrations in v1.1.0

Migration	What it does	Destructive?	Down script	Previous release still works?
001_initial_schema	All tables, keys, indexes, the audit trigger and the receipt sequence	—	None: the floor that rollbacks return to	—
002_cellphone_numbers	Rewrites every stored number to +27 form, blanks those that cannot be converted, adds CHECK constraints	**Yes**: changes data. Originals copied to a backup table first, in the same transaction	Restores the originals unless the row was edited since; drops the constraints	No: v1.0.x can save numbers the new CHECK refuses
003_account_recovery	Adds `recovery_codes`	No: additive	Drops the table	Yes: earlier releases ignore the table

Migration	What it does	Destructive?	Down script	Previous release still works?
004_user_lifecycle_and_retention	Adds offboarding, erasure, archive and anonymisation columns with CHECK constraints; back-fills archive dates	No data removed; adds constraints	Drops the columns and constraints	No: v1.0.x deactivates an account without the date the new CHECK needs

Two rules make these safe to apply and to undo. Each migration runs in its own transaction, so a failure part-way leaves nothing behind and is not recorded as applied. And a migration that changes existing data keeps a copy of what it changed until a later release removes it. Where a release must stay compatible with the one before it, the expand-then-contract pattern applies: add first, remove one release later. v1.1.0 is deliberately not backward compatible, because the feedback asked the database to enforce the new rules itself; Section 11.4 therefore says to undo the migrations before redeploying an older release.

9. Architecture and security

9.1 Architecture

The system keeps the layered architecture chosen in Task 1, behind a versioned REST API (`/api/v1`) with a single-page React application as the client. Dependencies point inward: routes depend on services, services on the domain and on repository interfaces, and nothing in the domain depends on anything else. Microservices were rejected in Task 1 for a single tightly coupled domain and one team, and nothing since has changed that.

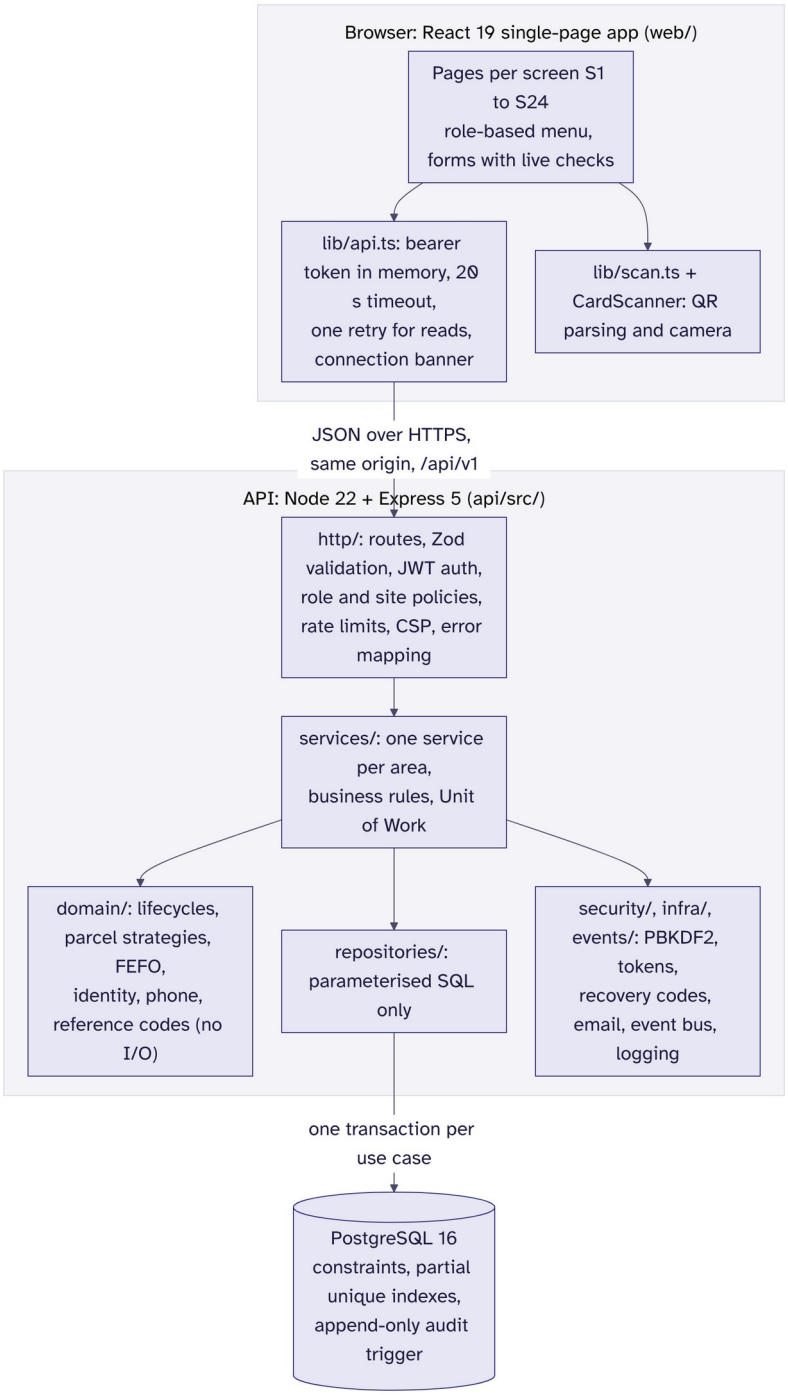


Figure 21. System architecture as built

9.2 Design patterns, and where to find them

Table 25. Design patterns applied

Pattern	Problem it solves here	Code
Repository	Services read and save without knowing SQL, so the rules read in one place	`api/src/repositories/`
Unit of Work	A collection, its stock deductions and its audit entry commit together or not at all	`UnitOfWork` in `repositories/index.ts`
Strategy	Parcel sizing by household size changes with funding; a new rule is a new class	`domain/parcel.ts`
Factory method	One place defines the reference-code format and its collision retry	`domain/referenceCode.ts`
Observer	Recording a collection or a withdrawal triggers emails and updates the service does not know about	`events/eventBus.ts`
Dependency injection	Tests swap the clock and mail sender without touching production code	`container.ts`
State machine	Household and event lifecycles allow only the transitions in Section 7.2	`domain/household.ts`, `domain/distributionEvent.ts`
Data Transfer Object	The identity hash and password hash never leave the API	`toPublicUser`, `lookupByReference`
Timeout and retry (new)	A request on a poor connection never hangs; reads retry once, writes never blindly	`web/src/lib/api.ts`
Idempotent write by constraint (new)	Repeating a collection after a lost answer cannot issue a second parcel	BR-04 index, `CollectionService`

9.3 Cloud architecture as deployed

Everything runs in the Azure South Africa North region, so data stays in the country and latency from Johannesburg is low. The diagram shows the services, what can reach what, and the protocol on each connection.

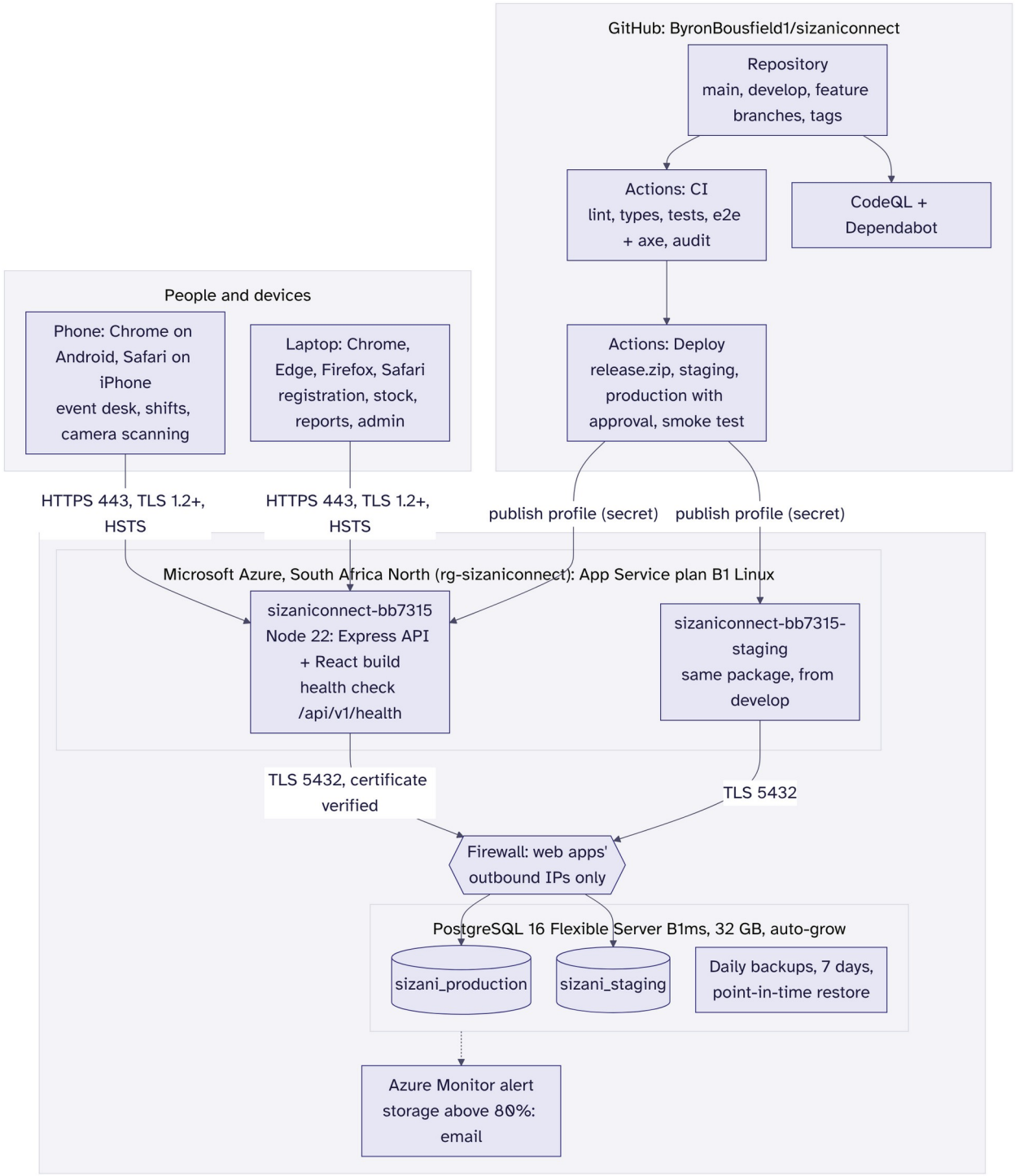


Figure 22. Deployment and cloud architecture (UML deployment view)

Table 26. Cloud services and network exposure

Service	Role	Exposure	Protocol
App Service plan B1 (Linux)	Hosts both web apps for one price	—	—
Web app sizaniconnect-bb7315	Production: Express API and the React build, from `main`	Public, HTTPS only, HSTS	HTTPS 443, TLS 1.2+

Service	Role	Exposure	Protocol
Web app sizaniconnect-bb7315-staging	Staging: the same package, from `develop`	Public, HTTPS only	HTTPS 443, TLS 1.2+
PostgreSQL 16 Flexible Server B1ms	Databases sizani_production and sizani_staging; 32 GB with auto-grow; 7-day backups with point-in-time restore	Public endpoint; firewall admits only the web apps' outbound addresses	TCP 5432, TLS required, certificate verified
Azure Monitor alert	Email when database storage passes 80%	Platform	—
GitHub Actions	CI, CodeQL, deployment with approval	Publishes with a publish profile held as a GitHub secret	HTTPS

The API is stateless: sessions are signed tokens and refresh tokens in the database, so a second instance can be added with no code change when NFR-05's load arrives.

9.4 Threats and controls

The Task 1 threat assessment (STRIDE) still stands. The table gives the control as built for each threat, where it is tested, and four threats added during Task 3.

Table 27. Threats and controls

ID	Threat	Control as built	Tested in
T-01	Credential stuffing	PBKDF2-SHA256 at 600,000 iterations with per-user salt; lock-out after five failures in 15 minutes; ten sign-ins a minute per address; 12-character minimum and a common-password list	`auth.test.ts`
T-02	Cross-site scripting	Strict CSP with no inline script; React output encoding; lint rule banning `dangerouslySetInnerHTML` and `innerHTML`; server-side validation; HttpOnly cookies	`households.test.ts`
T-03	SQL injection	Parameterised queries only; a lint rule fails the build on interpolated SQL; LIKE wildcards escaped	`households.test.ts`
T-04	One user reaching another's data	Donors see only their own records (another donor's receipt is a 404); coordinators change only their own site's households (fixed in v1.1.0)	`supply-admin.test.ts`, `households.test.ts`
T-05	Privilege escalation	Deny-by-default role policies on every route; the browser hides controls but the API decides	`auth.test.ts`
T-06	Database disclosure	ID numbers as a keyed hash; recovery codes as keyed hashes; TLS with certificate checking; firewall	`households.test.ts`
T-07	Exposed secrets	Secrets only in App Service settings and GitHub secrets; secret scanning with push protection	Repository settings
T-08	Cross-site request forgery	Bearer tokens for the API; refresh cookie SameSite=Strict, path-scoped, with an Origin check	`auth.test.ts`
T-09	Repudiation	Append-only audit log enforced by a trigger	`households.test.ts`

ID	Threat	Control as built	Tested in
T-10	Denial of service	1,000 requests a minute per address; bodies capped at 100 KB; reports capped at a month; pages capped at 100; statements stopped after 15 seconds	`rate-limit.test.ts`, `failures.test.ts`
T-11	Vulnerable dependency	`npm audit` gate in CI, CodeQL, Dependabot	CI
T-12	Enumerating the register	UUID keys and random reference codes	`domain.test.ts`
T-13	The only administrator locked out (new)	Recovery codes, reset links between administrators, audited break-glass command; the last administrator cannot be offboarded	`account-recovery.test.ts`
T-14	Personal data kept after it is needed (new, POPIA)	Offboarding and erasure; household anonymisation after 24 months	`retention.test.ts`
T-15	Malformed or hostile scan input (new)	The same parser in browser and API; 64-character cap; control characters stripped; foreign codes refused	`domain.test.ts`, `desk.spec.ts`
T-16	Personal data or tokens in logs (new)	Secrets redacted; request logs keep the path only	`logging.test.ts`

9.5 Authentication, sessions and recovery

An access token lasts 15 minutes and is held only in memory in the browser. A refresh token lasts 7 days in an HttpOnly, Secure, SameSite=Strict cookie, rotates on every use, and replaying an old one revokes the whole family of sessions. Authorisation is two checks on every request: the role (may this kind of user do this?) and the resource (may this user do it to this record?).

Table 28. Account recovery routes

Way back in	Who uses it	Lifetime	Safeguards
Email reset link	Anyone, when email is configured	30 minutes, single use	Same answer whether or not the account exists; ends other sessions
Recovery code	Anyone who created codes in advance	Until used or replaced	Ten codes, shown once, stored as keyed hashes; wrong codes count towards lock-out
Reset link from an administrator	Any active user, including another administrator	24 hours, single use	Audited; shown on screen when email is not configured
Break-glass command	Someone with access to the Azure portal, for an administrator	30 minutes, single use	Works only for active administrators; audited as BREAK_GLASS_RESET
Change password	A signed-in user	—	Needs the current password; signs out other devices; notice email

Transport: HTTPS only with HSTS for a year; TLS 1.2 or later. Headers: the CSP allows nothing but the site itself; `Permissions-Policy` allows the camera for this site only (for card scanning) and turns off the microphone, location, payment and USB; `X-Content-Type-Options`, `Referrer-Policy` and `Cache-Control: no-store` on every API answer.

9.6 POPIA

Table 29. POPIA conditions

Condition	How the system meets it	Status
Processing limitation	Only what eligibility, issue and reporting need: age bands not birth dates, ID numbers hashed, volunteers see no address or phone	Built
Retention limitation	Leavers offboarded and erased; archived households anonymised after 24 months	Built in v1.1.0
Security safeguards	Section 9.4, encryption in transit and at rest, least privilege, audit trail	Built
Data subject participation	Subject-access export of everything held about a household	Built
Information quality	Corrections through a coordinator, each one audited	Built
Accountability	The audit log shows who processed what and when; Sizani names an information officer	Built; officer is the client's decision
Openness, purpose specification	Privacy notice at registration and in volunteer induction	Recommended to the client: wording is the client's

10. Testing and quality

10.1 Test levels

Table 30. Test levels

Level	What it covers	Tool	Tests	Runs
Unit	Domain rules and security: parcel sizing, FEFO, lifecycles, ID numbers, cellphone parsing, card payloads and bad scans, failure classification, password hashing and tokens	Vitest	93	Every push
Integration	The real HTTP pipeline against a fresh PostgreSQL database per file: every route, role and resource check, two devices recording the same household at once, migrations rolled back and forward, account recovery, offboarding and erasure, the app put into each failure state, request logging	Vitest, supertest, PostgreSQL 16	116	Every push
Component	Sign-in and role navigation, the desk's scan handling, cellphone input, My account and recovery codes, time-outs and retries	Vitest, Testing Library	33	Every push
End-to-end and accessibility	Chromium driving the built app on a seeded database at 1280 px and 360 px: WCAG 2.1 AA checks on every main screen for every role, AC-09.1 to 09.7 including a real camera scan, recovery, offboarding, NFR-16, going offline, a lost save, a hung request	Playwright, axe-core	26	Every push, after the API and web jobs
Load	NFR-01 to NFR-03 on 5,050 households and about 19,000 collections	`scripts/load-test.ts`	5 operations	Before a release
Smoke	The health endpoint reports the commit just deployed and a working database	Deploy workflow	1	Every deployment

All 268 automated tests (209 API, 33 component and 26 end-to-end) pass on the v1.1.0 code, and the smoke test passes on every deployment. Each feedback item has at least one test that fails without the change, so the feedback cannot quietly come undone in a later release.

10.2 Coverage

Table 31. API test coverage, v1.1.0

Measure (API)	Result	Gate
Statements	96.9% (1,664 of 1,717)	70% (NFR-18)
Branches	86.7% (586 of 676)	—
Functions	96.4% (159 of 165)	—
Lines	96.9%	70%

10.3 Performance

The load test tops a seeded database up to 5,000 households with six months of weekly collections at six sites, then drives the API with many requests at once and reports the 50th and 95th percentile response times.

Table 32. Load test results (29 October 2026)

Requirement	Operation	Requests	At once	Errors	p50	p95	Target (p95)
NFR-01	Household search by name	1,000	25	0	329 ms	431 ms	2.0 s
NFR-01	Desk lookup by reference code	1,000	25	0	58 ms	91 ms	2.0 s
NFR-02	Record a collection with FEFO deduction	300	10	0	80 ms	647 ms	1.5 s
NFR-03	Monthly report on screen	20	2	0	57 ms	88 ms	8.0 s
NFR-03	Monthly report CSV export	20	2	0	49 ms	70 ms	20.0 s

Every target is met with at least a two-fold margin. The test ran on a 2-vCPU machine with the API and database together; Azure B1 and B1ms each have one virtual processor, and the margin is the allowance for that. Recording a collection spreads wider because ten desks drawing from the same batches wait on the row locks that stop two desks issuing the same last bag of maize. If name search ever nears its limit as the register grows, a trigram index on the name is the planned fix.

11. DevOps, deployment and rollback

11.1 Branching and version control

One GitHub repository. `main` is what production runs and every release on it is tagged (v1.0.0, v1.0.1, v1.1.0). `develop` is the integration branch and is deployed to staging. Work happens on short-lived branches cut from `develop` and named for what they do (`feature/…`, `fix/…`, `test/…`, `chore/…`); each is merged with a merge commit so the history shows what each branch delivered. A release is a merge of `develop` into `main` followed by a tag. Both long-lived branches are protected: a pull request and passing checks are required. Commit messages are short imperative summaries with a body that explains why, and they refer to the requirement, threat or feedback item concerned.

11.2 Continuous integration and deployment

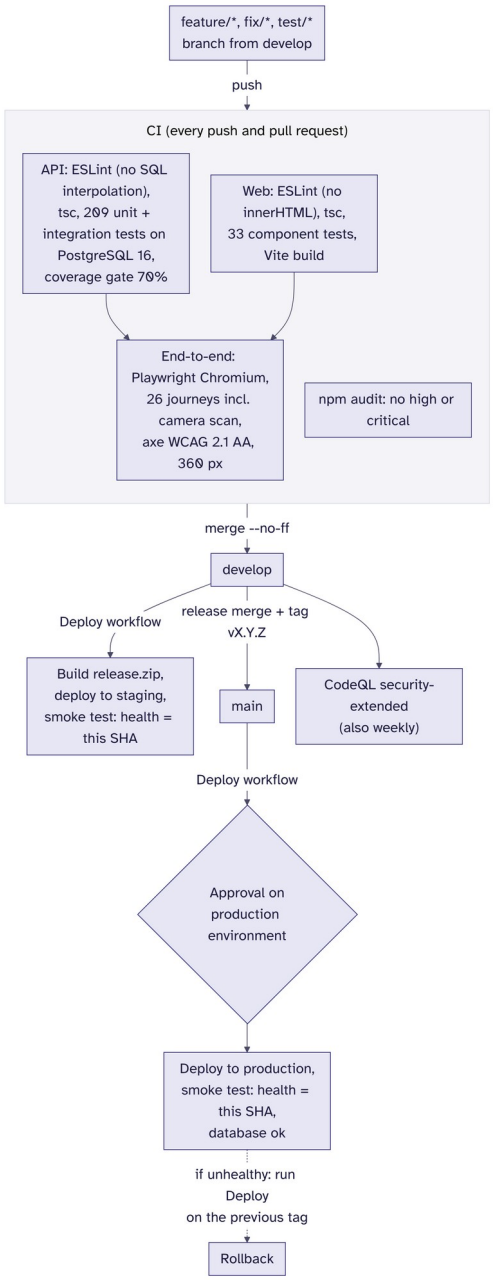


Figure 23. Branching, CI and CD as configured in GitHub Actions

Table 33. GitHub Actions workflows

Workflow	Job	What it does	Fails when
CI	API	ESLint (including the no-interpolated-SQL rule), TypeScript, unit and integration tests on a PostgreSQL 16 service container, coverage report	Any lint or type error, failing test, or coverage below 70%
CI	Web	ESLint (banning `innerHTML`), TypeScript, component tests, production build	Any error or failing test
CI	End-to-end and accessibility	Builds both, seeds a fresh database, runs Playwright with axe-core at desktop and phone sizes; ffmpeg supplies the fake camera feed	Any failing journey or accessibility violation
CI	Dependency audit	`npm audit` of production dependencies	Any high or critical advisory
Deploy	Build release	Reruns CI, then builds one self-contained package	CI fails
Deploy	Staging	Deploys `develop`, then polls `/api/v1/health` until it reports this commit and a working database	Smoke test times out
Deploy	Production	Waits for approval on the `production` environment, deploys `main` or a chosen release tag, smoke test	No approval, or smoke test fails
CodeQL	Analyse	Security-extended queries on every push to `develop` and `main`, and weekly	A new alert

11.3 Environments and rollout

Table 34. Environments

Environment	Where	Deployed from	Data
Local	Docker Compose (app and PostgreSQL), or Node 22 and PostgreSQL 16 directly	Working copy	Seeded synthetic data
CI	GitHub-hosted runners with a PostgreSQL service container	Every push	A fresh database per test file; seeded data for end-to-end
Staging	Web app sizaniconnect-bb7315-staging, database sizani_staging	`develop`, automatically	Synthetic demo data
Production	Web app sizaniconnect-bb7315, database sizani_production	`main` or a release tag, after approval	Synthetic demo data for the module; live data after handover

Rollout of a release, as done for v1.1.0:

1. Merge the release into `develop`; CI and the end-to-end suite pass; staging updates itself and its smoke test passes.
2. Check the release on staging: sign in as each demo role and run the event desk once.

3. Merge `develop` into `main` and tag the release. CI runs again on `main`.
4. Approve the production deployment in GitHub. The new build starts, applies its migrations under a lock, and the smoke test waits until the health check reports the new commit and a working database.
5. Watch the log stream and the health check for 30 minutes during which nobody is distributing.

11.4 Rollback

Table 35. Rollback procedures

Situation	What to do
A release misbehaves and its migrations are backward compatible	In GitHub, run the Deploy workflow on the previous release tag and approve it. The tag is rebuilt, tested, deployed and smoke-tested.
A migration fails while deploying	Nothing to undo in the database: the migration rolled back in its own transaction and is not recorded. The new build refuses to start and the smoke test fails, so the old build keeps running. Roll the application back as above, fix the migration, release again.
A migration that succeeded must be undone (destructive change, or going from v1.1.0 back to v1.0.x)	1. Outside distribution hours, note the time (UTC) as a restore point. 2. Add the app setting `SKIP_MIGRATIONS=true`. 3. In the web app's SSH console run `node dist/db/cli.js status`, then `node dist/db/cli.js rollback --to 001_initial_schema.sql`, then `status` again. Each step is one transaction; a failing step stays applied and nothing after it is attempted. 4. Redeploy the release that matches the schema. 5. Remove `SKIP_MIGRATIONS`. 6. Check the health endpoint and sign in.
Data damaged beyond what a down script can repair	Point-in-time restore of the PostgreSQL server to the noted time (backups kept 7 days), allow the web apps through the new server's firewall, change the connection string, redeploy the matching release. About an hour; NFR-07 allows four.

Erasures and anonymisations are never brought back by a rollback: removing personal data is meant to be permanent. CI rehearses the rollback of every migration on a populated database before every deployment and checks that the schema and row counts come back identical.

11.5 When something fails in production

Table 36. Failure runbook

Health status	Meaning	What people see	What to do
ok (200)	Database reachable and writable	Normal service	Nothing
degraded (200), database read-only	Storage full: Azure switched the server to read-only above 95%	Pages load; every save answers STORAGE_FULL; sessions end within 15 minutes	Raise the storage size in the portal. Saves resume within a minute; no restart. Auto-grow should prevent this

Health status	Meaning	What people see	What to do
down (503), database unreachable	Server restarting or failing over, firewall changed, password out of step	DATABASE_UNAVAILABLE and the yellow connection banner	Usually clears by itself. After five minutes: check the server is Ready, the firewall still lists the web apps' outbound addresses, and the log stream

12. Running costs

Costs are in South African Rand a month, excluding VAT, for the South Africa North region, using the same list-price estimates as Task 1 so the two can be compared. Three Task 1 components have dropped out because of the decisions in Section 3.2: Static Web Apps (the front end is served by the App Service), Key Vault (secrets are app settings) and Blob Storage (uploads deferred). They come back only if those decisions are reversed. The system as deployed today is the 100-user column.

Table 37. Monthly running cost at three scales

Component	100 users	R	1,000 users	R	10,000 users	R
Web app hosting (both apps on one plan)	App Service B1	290	App Service S1	1,150	P1v3, 2 instances	5,800
Database	PostgreSQL B1ms, 32 GB	230	B2s, 64 GB	900	D2ds_v4, 256 GB	2,600
Backups	Included, 7 days	0	Geo-redundant	180	Geo-redundant	520
Monitoring	Log stream, health check, one alert	0	Application Insights, 15 GB	300	Application Insights, 60 GB	1,200
Data transfer	About 10 GB	40	About 80 GB	250	About 400 GB	1,100
Email	Free tier	0	Paid tier	450	Paid tier	900
Firewall at the edge	Not needed	0	Not needed	0	Front Door with WAF	1,600
Domain	azurewebsites.net address	0	Own domain	25	Own domain	25
Total a month		**560**		**3,255**		**13,745**
Per user a month		5.60		3.26		1.37

The deployed system costs about R560 a month, which the Azure for Students credit covers during the module. Storage auto-grow adds about R2.50 a gigabyte a month, and only if it is ever used. The resource group should be deleted once the module is marked, unless the client takes the system over.

Table 38. Two-year cost projection

Period	Users	Households	Tier	R a month	R for the period
Year 1, Q1	100	1,400	Entry (B1, B1ms)	560	1,680
Year 1, Q2	250	3,000	Entry	560	1,680
Year 1, Q3	400	5,000	Entry, more storage	720	2,160
Year 1, Q4	600	7,000	Mid (S1, B2s)	3,255	9,765
Year 1					**15,285**
Year 2, Q1	800	9,000	Mid	3,255	9,765
Year 2, Q2	1,000	11,000	Mid	3,255	9,765

Period	Users	Households	Tier	R a month	R for the period
Year 2, Q3	1,250	14,000	Mid, more storage	3,700	11,100
Year 2, Q4	1,500	17,000	Mid, second instance	4,900	14,700
Year 2					**45,330**
Two years					**60,615**

The two-year total is about R6,900 lower than Task 1 projected (R67,515), almost all of it from serving the front end from the App Service and keeping secrets in its settings. As in Task 1, the step comes when all six sites are live and the entry tier can no longer carry a distribution day; and a registered non-profit should apply for Microsoft’s non-profit Azure grant, which would cover most of this.

13. Change management and handover

The adoption plan from Task 1 stands: a phased pilot at the site with the most engaged coordinator, a parallel run with paper as a fallback, cutover once the board signs off the first system-generated monthly report, and the donor portal last. What has changed is that the materials the plan depends on now exist.

Table 39. Handover materials

Task 1 commitment	Status in v1.1.0
User guide	Done: 55 pages, every feature by role with annotated screenshots, troubleshooting, FAQ and technical setup
Quick-reference card for each site	Done: the event desk quick card, user guide Appendix B, one page to print and laminate
Short screen recordings for training	Done: the walkthrough video, about four and a half minutes, covering every role
Administrator guide and runbook (backup, restore, incidents, releases)	Done: user guide Section 8 and Appendix A, and `docs/DEPLOYMENT.md` sections 5 to 7
Infrastructure as code	Done: `infra/azure-setup.sh` rebuilds the whole environment and is safe to re-run
Architecture decision record	Done: Section 3.2 of this report and `docs/CHANGES-FROM-TASK1.md`
Data migration of the existing register	Not started: needs the client's real register
Credentials handed to the client's information officer	At handover: the secrets live only in the App Service settings and can be regenerated by re-running the setup script

Two things in the system itself make adoption easier than Task 1 assumed. A volunteer's whole job at the desk is one field and one button, and a bad scan says what to do next, so the one-page card is enough training. And a coordinator whose phone loses signal is told plainly whether the collection was saved and that pressing the button again is safe, which removes the main reason a busy site would go back to paper. Offline capture remains the first candidate for the next phase.

Table 40. Adoption risks, updated

Risk	Response now
Coordinators go back to paper during a busy event	Paper fallback allowed in the pilot with same-day capture; the desk now handles poor signal safely
The only trained administrator leaves	Two administrators recommended; recovery codes; reset links between administrators; break-glass; offboarding warns when a site loses its coordinator
Personal data kept longer than allowed	Data retention screen lists what is due; erasure and anonymisation are one audited action each
Cloud cost exceeds the budget	One alert already set on storage; add a budget alert at 80% and 100% when the client takes over billing

14. Reflection and limitations

14.1 What is not done, and what would come next

- **Usability test with real volunteers (NFR-08).** The three-minute target needs five untrained people at a real desk. This is the first thing to do with a real client.
- **Firefox and Safari in the automated tests (NFR-11).** Playwright can run both; adding them to CI is a configuration change.
- **Attaching delivery notes (US-12).** Needs Blob Storage and an upload screen; deferred because no other story depends on it.
- **Offline capture.** Out of scope in Task 1 and still the most valuable next feature for sites with poor signal.
- **Private networking for the database.** Blocked on the student subscription; no code change is needed when it becomes possible.
- **A real client.** Every requirement here was derived from desk research for a simulated client. With a real one, Section 4 would be re-validated in workshops before anything else.

14.2 What I learned

Working alone made planning matter more, not less. Every hour spent on one thing was an hour not spent on another, and the plan in Section 5 is the result of learning that schema changes in particular must be done one after another, never side by side. Testing the system in the states it will really fail in (an unreachable database, a read-only one, a connection dropped mid-save, a camera feed from a creased card) was more work than mocking those failures, but it is the only reason I can say with confidence how the system behaves when they happen. And writing the user guide was the best test of all: explaining each screen to a non-technical reader exposed two real security problems that every automated test had missed, and running the browser tests over and over exposed a third, smaller fault that a single run had hidden, because no test had been written to look for them.

References

Fowler, M. 2002. *Patterns of enterprise application architecture.* Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. 1994. *Design patterns: elements of reusable object-oriented software.* Reading, MA: Addison-Wesley.

Grassi, P.A., Fenton, J.L., Newton, E.M. et al. 2017. *Digital identity guidelines: authentication and lifecycle management.* NIST Special Publication 800-63B. Gaithersburg, MD: National Institute of Standards and Technology.

ISO/IEC. 2015. *ISO/IEC 18004:2015 Information technology: automatic identification and data capture techniques: QR Code bar code symbology specification.* Geneva: International Organization for Standardization.

Microsoft. 2026. *Azure App Service documentation.* [Online]. Available at: https://learn.microsoft.com/azure/app-service/ [Accessed 29 October 2026].

Microsoft. 2026. *Azure Database for PostgreSQL flexible server documentation.* [Online]. Available at: https://learn.microsoft.com/azure/postgresql/flexible-server/ [Accessed 29 October 2026].

OWASP Foundation. 2021. *OWASP Top 10: 2021.* [Online]. Available at: https://owasp.org/Top10/ [Accessed 29 October 2026].

South Africa. 2013. *Protection of Personal Information Act 4 of 2013.* Government Gazette, 37067, 26 November 2013.

W3C. 2018. *Web Content Accessibility Guidelines (WCAG) 2.1.* W3C Recommendation, 5 June 2018. [Online]. Available at: https://www.w3.org/TR/WCAG21/ [Accessed 29 October 2026].

Annexure A. Declaration of Authenticity

I declare that:

1. I have been informed about the seriousness of acts of plagiarism.
2. I understand what plagiarism is.
3. I am aware that The Independent Institute of Education (IIE) has a policy regarding plagiarism and that it does not accept acts of plagiarism.
4. I am aware that the Intellectual Integrity Policy and the Student Code of Conduct prescribe the consequences of plagiarism.
5. I am aware that referencing guides are available in my student handbook or equivalent and in the library, and that following them is a requirement for successful completion of my programme.
6. I am aware that should I require support or assistance in using referencing guides to avoid plagiarism, I may speak to the lecturers, the librarian, or the campus ADC or Campus Co-ordinator for clarification.
7. I am aware of the consequences of plagiarism.

I further declare that the work submitted is my own, that all sources used have been acknowledged, and that the client, Sizani Community Food Network NPC, is a simulated case-study client: no real organisation was engaged, and all people, households and donors in the system's data are synthetic.

Name	Student number	Signature	Date
Byron Bousfield	ST10445541		30 October 2026

Annexure B. Peer evaluation

Group 13 has one member. No other students were allocated to the group, the lecturer was informed by email, and the project was carried out alone from Task 1 to Task 3. There are therefore no peers to evaluate and no peer evaluations of me. In place of the evaluation, the table records who did what.

Table 41. Contribution record

Member	Work carried out	Share
Byron Bousfield, ST10445541	Task 1: requirements, analysis and design documentation, prototype. Task 2: front end, back end, database, Azure hosting, CI/CD, presentation. Task 3: every change in this report, tests, user guide, walkthrough video, final report, EXPO material. Weekly registers throughout.	100%

Name	Signature	Date
Byron Bousfield		30 October 2026

Annexure C. User guide

The user guide is a separate document, *SizaniConnect User Guide*, included after this report in the combined submission. It is written for the people who use the system, not for developers, and every instruction is illustrated with an annotated screenshot of the live system.

Table 42. User guide contents

Chapter	Contents
1. About this guide	Who it is for and how to read the instructions
2. Getting started	What you need, opening the system, activating an account, signing in, finding your way, My account and recovery codes, a forgotten password
3. Site coordinators	Overview, registering a household, printing a household card, finding a household, running a distribution day, approving a second parcel, shifts
4. The event desk	Recording a collection, scanning with the camera, when a card cannot be used, when the signal drops
5. Volunteers	Signing up, checking in, withdrawing
6. Operations managers	Verifying households, suspending and archiving, scheduling events, donations and receipts, stock, the monthly report
7. Donors	Your giving and receipts
8. Administrators	Adding users, helping someone locked out, when someone leaves, data retention, sites, the audit log
9. When something goes wrong	Every message and what to do; frequently asked questions
10. Glossary	
Appendix A	Technical setup: requirements, Docker, running without Docker, configuration, Azure deployment, emergency administrator access, backups and rollback
Appendix B	Event desk quick card, to print

Annexure D. EXPO materials

The EXPO showcases SizaniConnect as a working system, not as slides. The PowerPoint gives a two-minute story for visitors; the display board stands beside a live demonstration; and the props let a visitor play a volunteer at the event desk.

Table 43. EXPO materials

Item	Description
PowerPoint	*SizaniConnect EXPO* deck: the problem, the solution in one picture, a live-demo slide, what the feedback changed, how it is built and tested, and the results. Included after the user guide in the combined submission and downloadable as .pptx.
Display board	A1 portrait poster: problem, solution, the event desk on a phone, the three feedback changes visitors can try, architecture, test and performance figures, and a QR code to the live system.
Live demonstration	A laptop signed in as an operations manager and a phone signed in as a volunteer on today's open event at Diepsloot, both on the production system.
Props	Printed household cards (valid, creased, and one "foreign" QR code) for visitors to scan at the desk; the event desk quick card; printed copies of the user guide.
Demonstration script	1. Scan a valid card: eligible, record, hand over. 2. Scan it again: already collected. 3. Scan the creased card: explained, nothing sent. 4. Switch the phone to flight mode and record: the yellow banner and a plain message; back online, record again. 5. On the laptop: the monthly report and the audit trail of what the visitor just did.

Annexure E. Glossary

Table 44. Glossary

Term	Meaning
Anonymise	Remove the personal data from a record but keep the record, so counts and history still add up
Break-glass	An emergency way in for an administrator, used only when every normal route has failed, and always audited
Collection	The record that a household received a parcel at a distribution event
E.164	The international format for phone numbers: +27 followed by nine digits for a South African cellphone
Erase	Remove a person’s personal data for good after they have been offboarded
Expand and contract	Changing a database in two releases: add the new thing first, remove the old thing one release later, so either release can run on the schema
FEFO	First expiry, first out: stock closest to expiry is issued first
Health check	An address the platform and the deployment pipeline call to see whether the system and its database are working
Luhn mod 32	A check-character method that catches any single wrong character and most swapped pairs in a code
Migration	A numbered script that changes the database structure; its down script undoes it
NPC	Non-profit company, under the Companies Act 71 of 2008
Offboard	Stop a person’s access when they leave, keeping what they did
POPIA	The Protection of Personal Information Act 4 of 2013
Recovery code	One of ten single-use codes a user keeps somewhere safe to get back in without email
Reference code	The short code that identifies a household, such as SZN-2026-7K2QX9, printed on its card under a QR code
Release candidate (RC)	A point at which the work is frozen and checked before the next stage starts
Smoke test	A quick check straight after a deployment that the system is up and is the version just deployed
Stock batch	A consignment of one stock item received together with one expiry date
WIL	Work Integrated Learning